



US010878817B2

(12) **United States Patent**
Toplyn

(10) **Patent No.:** **US 10,878,817 B2**

(45) **Date of Patent:** ***Dec. 29, 2020**

(54) **SYSTEMS AND METHODS FOR
GENERATING COMEDY**

(56) **References Cited**

U.S. PATENT DOCUMENTS

(71) Applicant: **Twenty Lane Media, LLC**, Rye, NY
(US)

5,708,822 A * 1/1998 Wical G06F 40/253
704/1

(72) Inventor: **Joseph E. Toplyn**, Harrison, NY (US)

7,376,551 B2 5/2008 Powell et al.
(Continued)

(73) Assignee: **Twenty Lane Media, LLC**, Rye, NY
(US)

FOREIGN PATENT DOCUMENTS

WO 2018045081 A1 3/2018

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 0 days.

OTHER PUBLICATIONS

Excerpts from Joe Toplyn, "Comedy Writing for Late-Night TV:
How to Write Monologue Jokes, Desk Pieces, Sketches, Parodies,
Audience Pieces, Remotes, and Other Short-Form Comedy," pub-
lished 2014 by Twenty Lane Media, LLC of Rye, NY.

(Continued)

(21) Appl. No.: **16/831,526**

(22) Filed: **Mar. 26, 2020**

Primary Examiner — Douglas Godbold

(74) *Attorney, Agent, or Firm* — Law Office of Paul B.
Johnson; Paul B. Johnson

(65) **Prior Publication Data**

US 2020/0227032 A1 Jul. 16, 2020

Related U.S. Application Data

(63) Continuation-in-part of application No. 16/250,972,
filed on Jan. 17, 2019, now Pat. No. 10,642,939.
(Continued)

(51) **Int. Cl.**
G10L 15/22 (2006.01)
G10L 15/18 (2013.01)
(Continued)

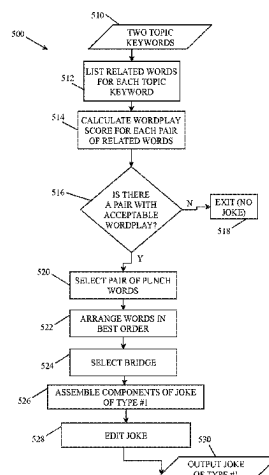
(52) **U.S. Cl.**
CPC **G10L 15/22** (2013.01); **G06F 40/40**
(2020.01); **G10L 13/00** (2013.01);
(Continued)

(58) **Field of Classification Search**
CPC G10L 15/22
See application file for complete search history.

(57) **ABSTRACT**

Methods for generating jokes include coupling server(s)
with database(s) having words stored therein; receiving text
at the server(s) from an external source communicatively
coupled with the server(s) through a telecommunications
network; in response to receiving the text at the server(s):
selecting one or more topic keywords of the topic sentence
using the server(s); generating one or more punch words
with the server(s) using words stored in the database related
to the topic keyword(s); adding bridges to the punch word
(s), using the server(s), to generate one or more jokes;
communicating a signal to a first computing device through
the telecommunications network using the server(s); and in
response to receiving the signal at the first computing
device, displaying or speaking the joke(s) using the first
computing device. Systems for generating jokes include
networked computer components configured to carry out the
methods. The methods/systems may also be used for recog-
nizing jokes.

12 Claims, 15 Drawing Sheets



Related U.S. Application Data

- (60) Provisional application No. 62/634,845, filed on Feb. 24, 2018.
- (51) **Int. Cl.**
G10L 15/30 (2013.01)
G10L 13/00 (2006.01)
G06F 40/40 (2020.01)
G10L 15/08 (2006.01)
- (52) **U.S. Cl.**
 CPC **G10L 15/1815** (2013.01); **G10L 15/30** (2013.01); **G10L 2015/088** (2013.01); **G10L 2015/223** (2013.01)

(56) References Cited**U.S. PATENT DOCUMENTS**

7,487,081	B2	2/2009	Shimomura et al.	
9,015,033	B2	4/2015	De Andrade Barbosa et al.	
2004/0015344	A1 *	1/2004	Shimomura	G10L 15/22 704/200
2004/0088327	A1 *	5/2004	Shimomura	G06F 40/56
2006/0067352	A1	3/2006	John et al.	
2006/0200734	A1 *	9/2006	Goradia	G06F 16/20 715/209
2012/0117101	A1 *	5/2012	Unruh	G06F 3/0237 707/765
2012/0316865	A1 *	12/2012	Miyazaki	G06F 40/30 704/9
2017/0060849	A1	3/2017	Allen et al.	
2017/0371859	A1 *	12/2017	Allen	G06F 40/30
2018/0226067	A1 *	8/2018	Dhoolia	G10L 15/18
2019/0193273	A1 *	6/2019	Favis	B25J 11/001
2019/0266250	A1	8/2019	Toplyn	
2020/0042596	A1 *	2/2020	Ravi	G06F 40/30

OTHER PUBLICATIONS

STANDUP: System to Augment Non-Speakers' Dialogue Using Puns, published online at <https://www.abdn.ac.uk/ncs/departments/>

computing-science/standup-315.php at least as early as Feb. 23, 2018, available online at <https://www.abdn.ac.uk/ncs/departments/computing-science/standup-315.php>, last visited Dec. 26, 2018.

Computational Humor Wikipedia page, published online at https://en.wikipedia.org/wiki/Computational_humor at least as early as Feb. 23, 2018, available online at https://en.wikipedia.org/wiki/Computational_humor, last visited Dec. 26, 2018.

JAPE Summary, published online at <http://www.nlg-wiki.org/systems/JAPE> at least as early as Feb. 23, 2018, available online at <http://www.nlg-wiki.org/systems/JAPE>, last visited Dec. 26, 2018.

HAHAcronym: Humorous Agents for Humorous Acronyms, published online at https://www.researchgate.net/publication/249929592_HAHAcronym_Humorous_Agents_for_Humorous_Acronyms at least as early as Feb. 23, 2018, available online at https://www.researchgate.net/publication/249929592_HAHAcronym_Humorous_Agents_for_Humorous_Acronyms, last visited Dec. 26, 2018.

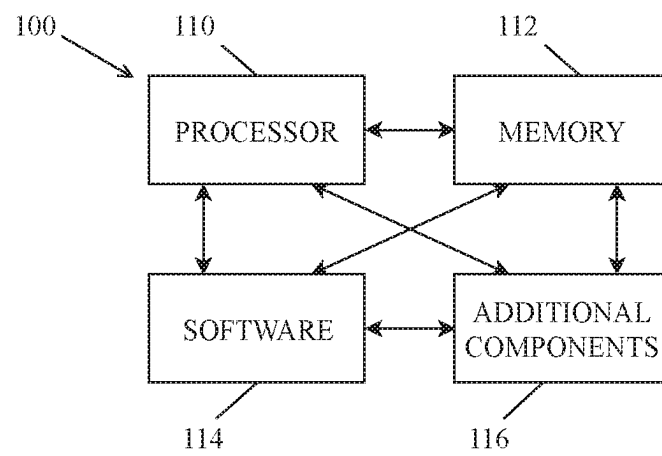
Chloe Kiddon and Yuriy Brun, "That's What She Said: Double Entendre Identification," Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: shortpapers, pp. 89-94, Portland, Oregon, Jun. 19-24, 2011, published online at <https://people.cs.umass.edu/~brun/pubs/pubs/Kiddon11.pdf> at least as early as Feb. 23, 2018, available online at <https://people.cs.umass.edu/~brun/pubs/pubs/Kiddon11.pdf>, last visited Dec. 26, 2018.

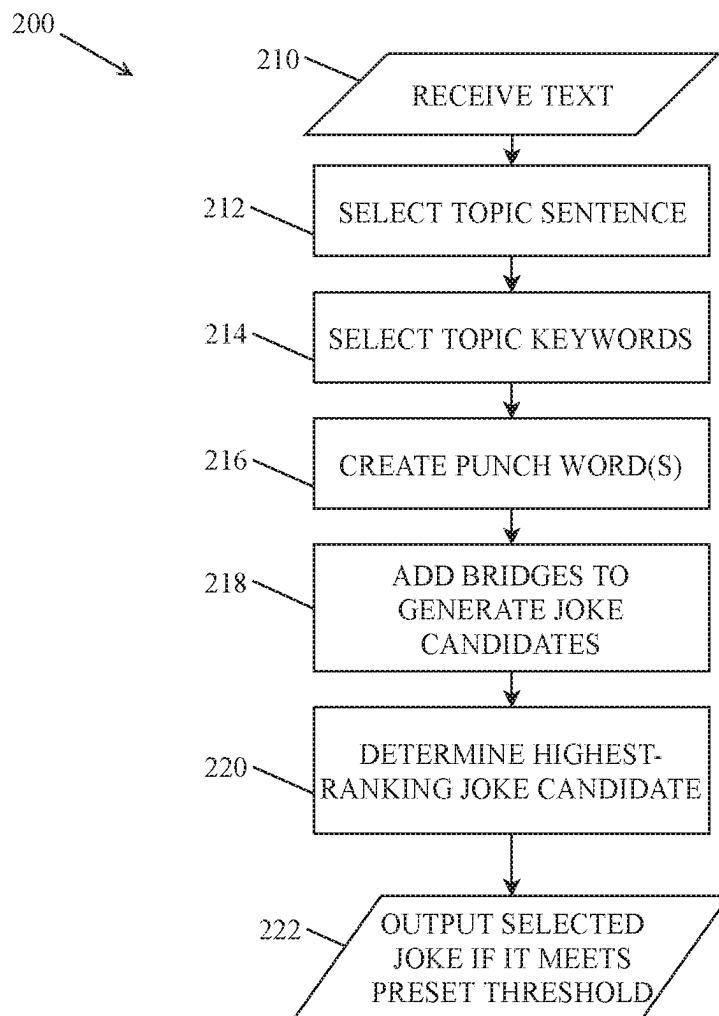
Becky Ferreira, "Joke Telling Robots Are the Final Frontier of Artificial Intelligence," published online Mar. 15, 2016 at https://motherboard.vice.com/en_us/article/z43nke/joke-telling-robots-are-the-final-frontier-of-artificial-intelligence, available online at https://motherboard.vice.com/en_us/article/z43nke/joke-telling-robots-are-the-final-frontier-of-artificial-intelligence, last visited Jan. 11, 2019.

Alexander Martin, "IBM's Watson to Help Pepper Robot Learn Faster," published online Jan. 7, 2016 at <https://blogs.wsj.com/japanrealtime/2016/01/07/ibms-watson-to-help-pepper-robot-learn-faster>, available online at <https://blogs.wsj.com/japanrealtime/2016/01/07/ibms-watson-to-help-pepper-robot-learn-faster>, last visited Jan. 11, 2019.

International Search Report and Written Opinion of the International Searching Authority for PCT/US19/19326, which is a PCT application of the immediate parent of the instant application, dated May 2, 2019.

* cited by examiner

**FIG. 1**

**FIG. 2**

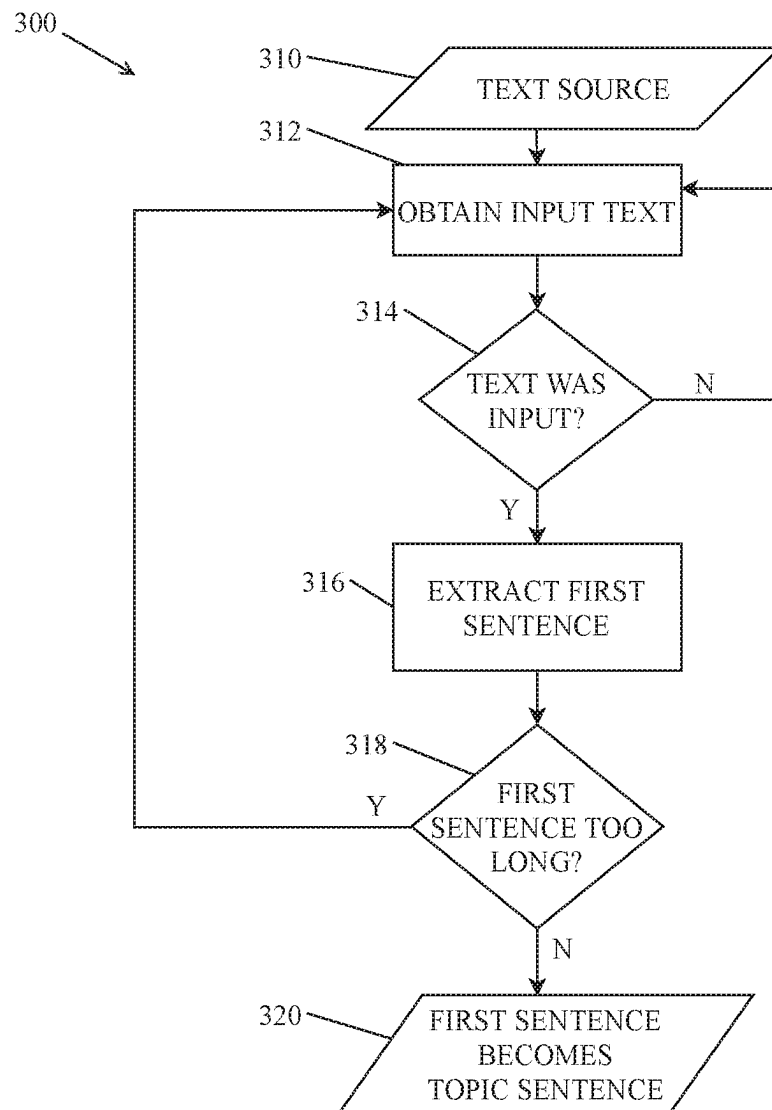


FIG. 3

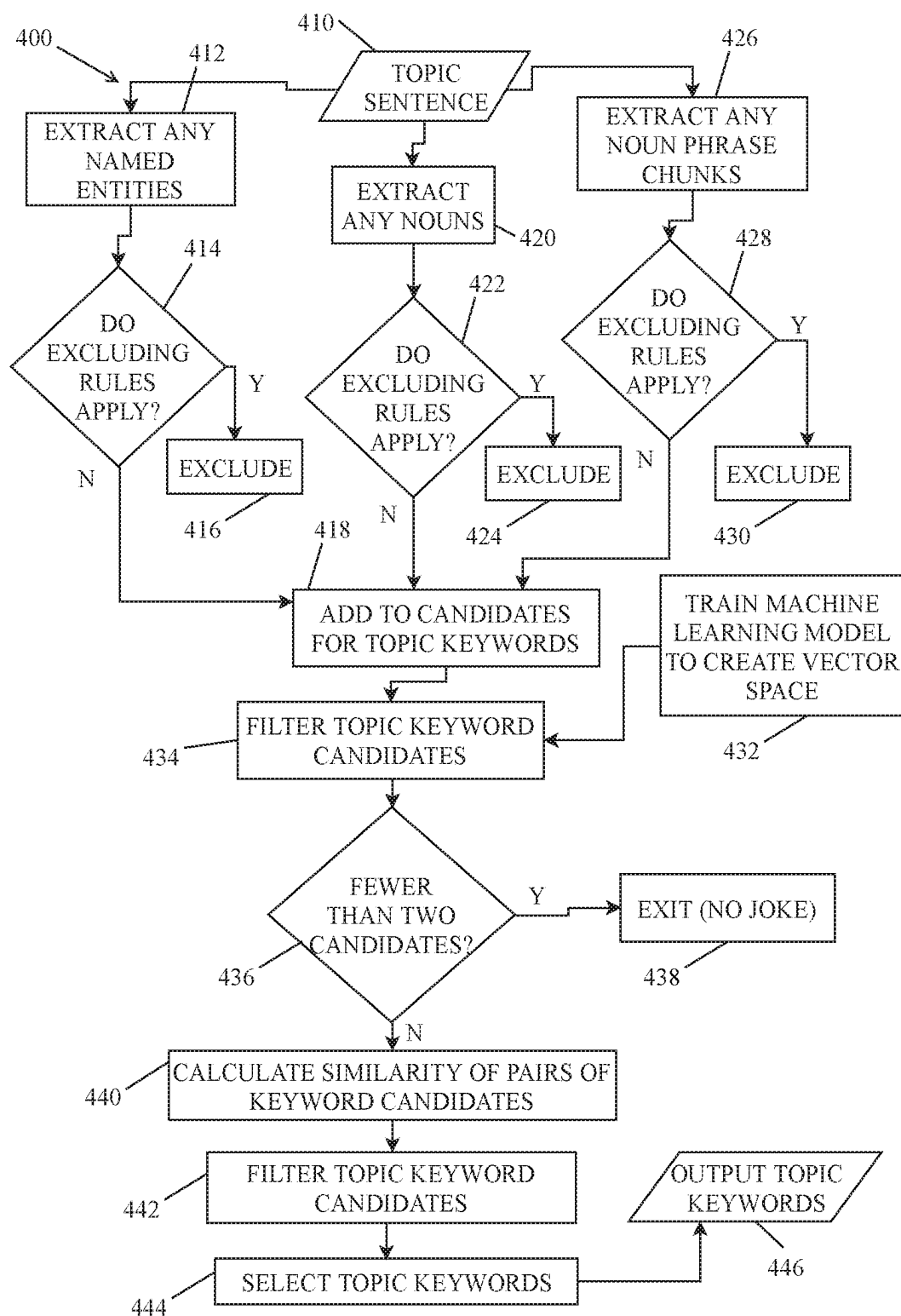
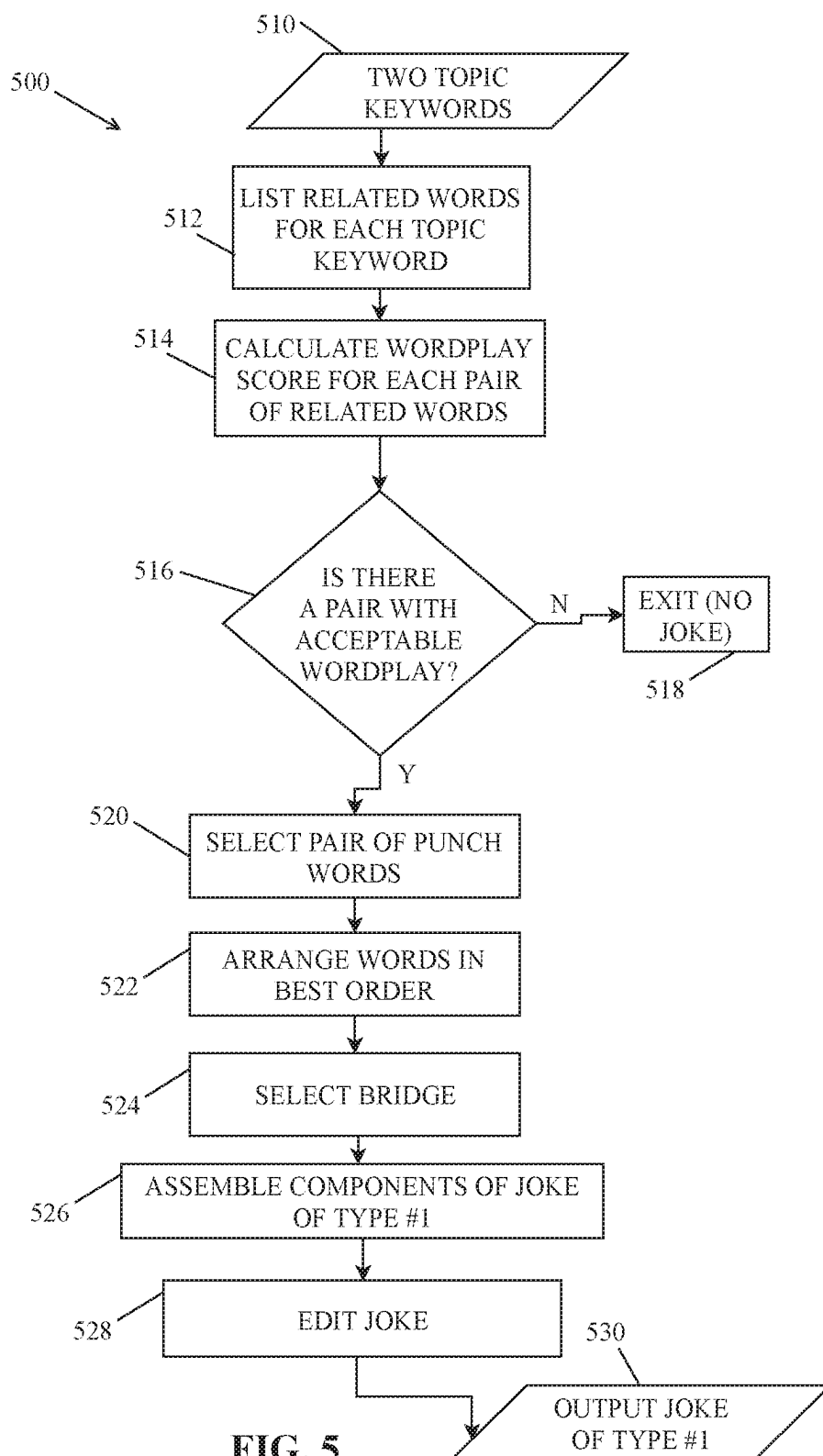


FIG. 4



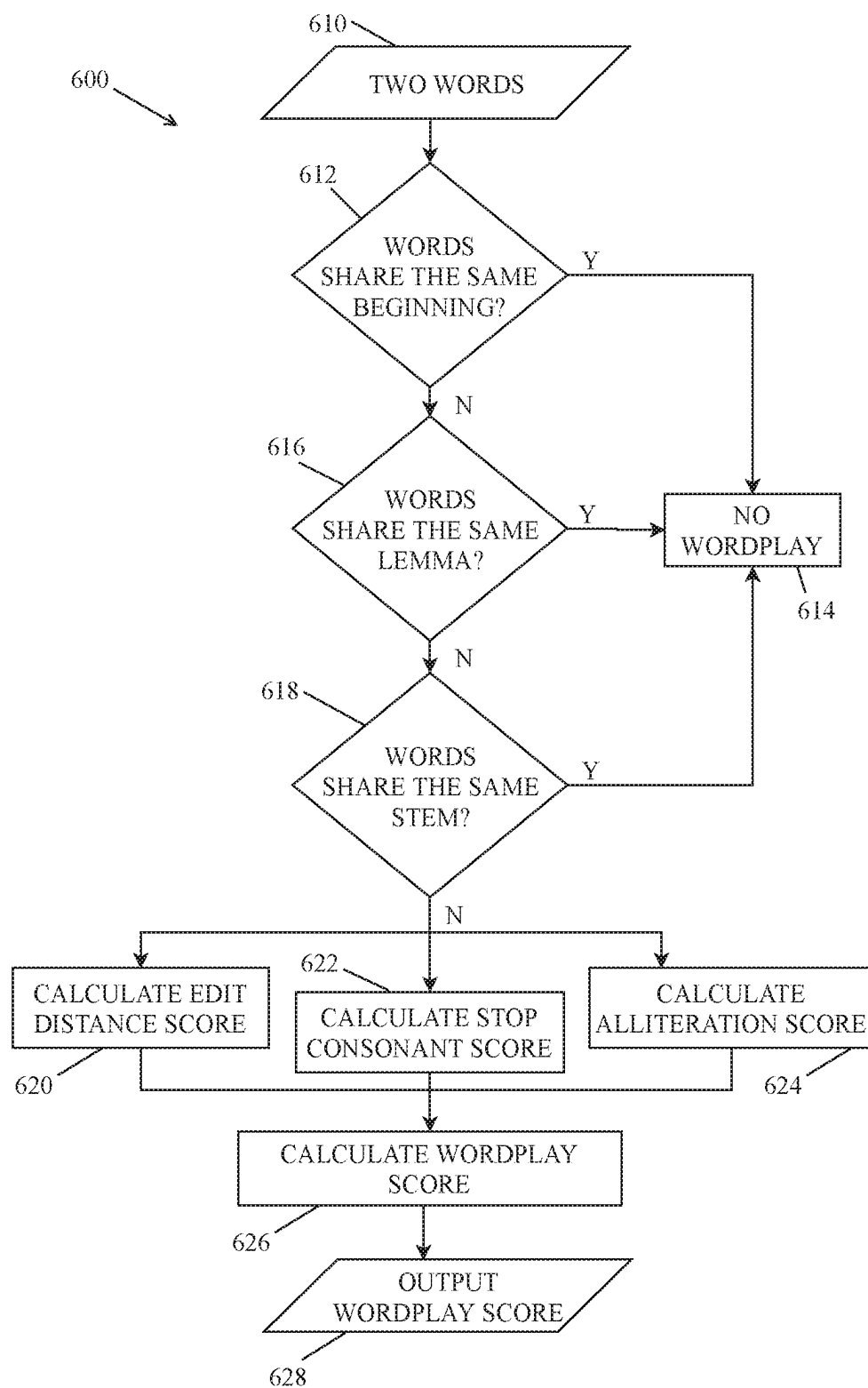


FIG. 6

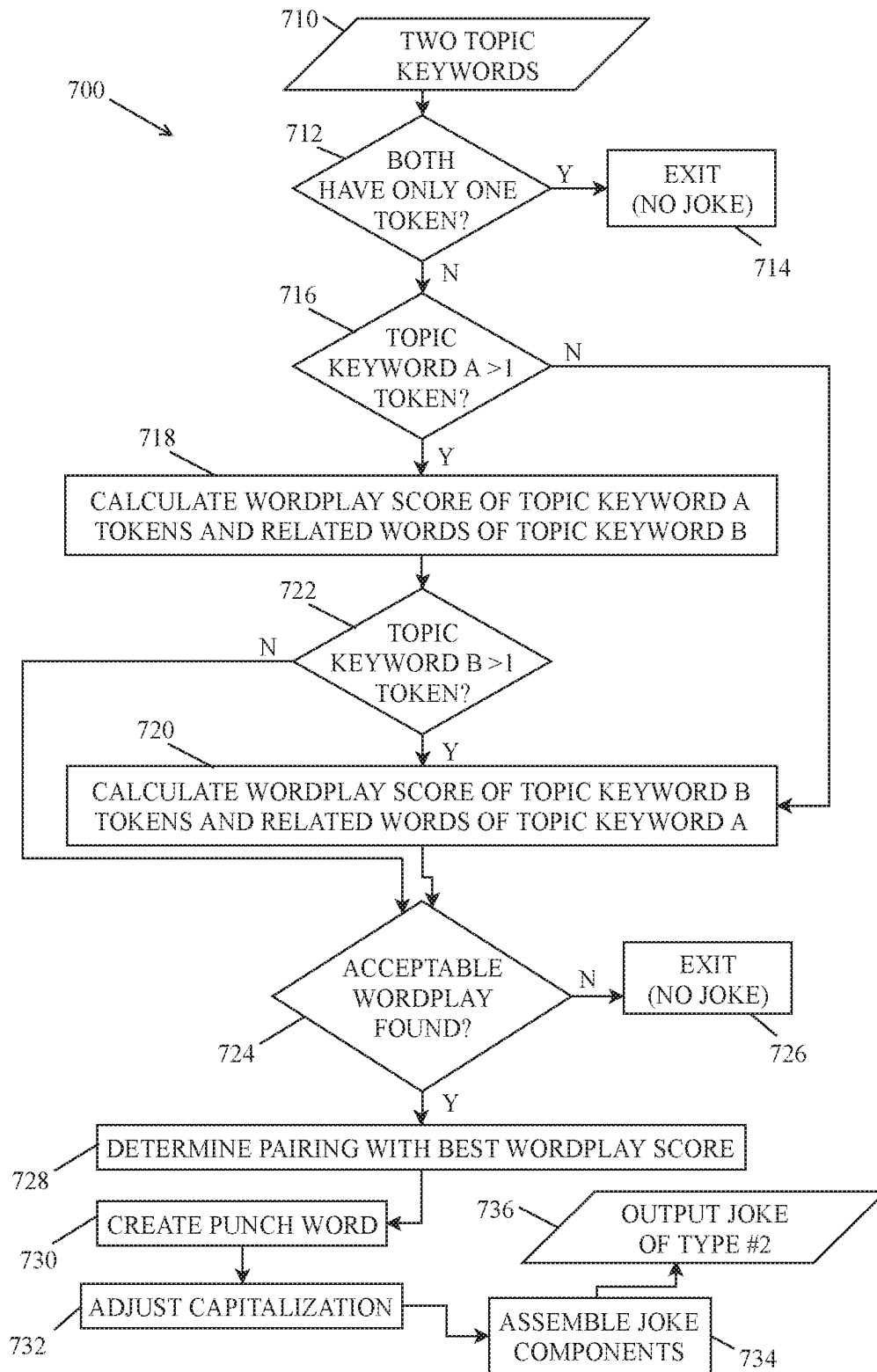
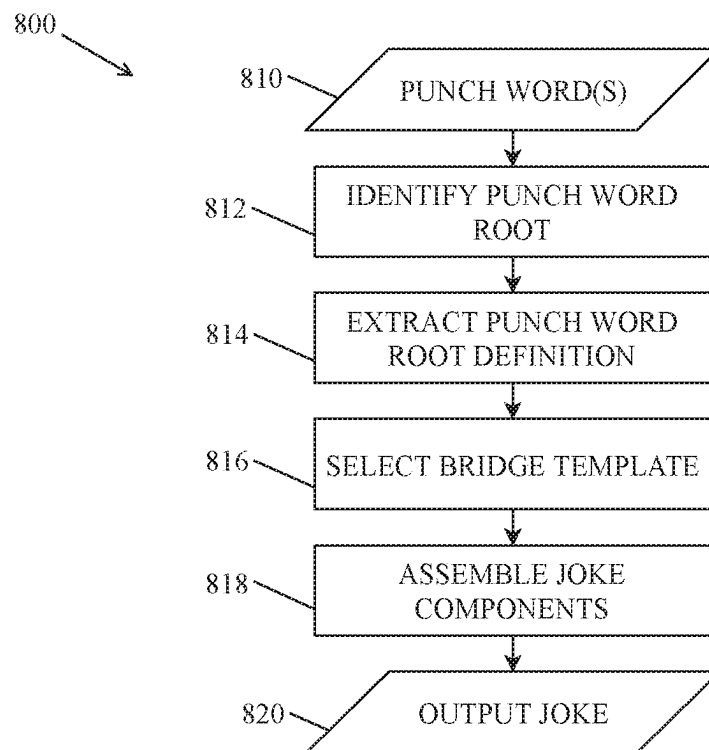


FIG. 7

**FIG. 8**

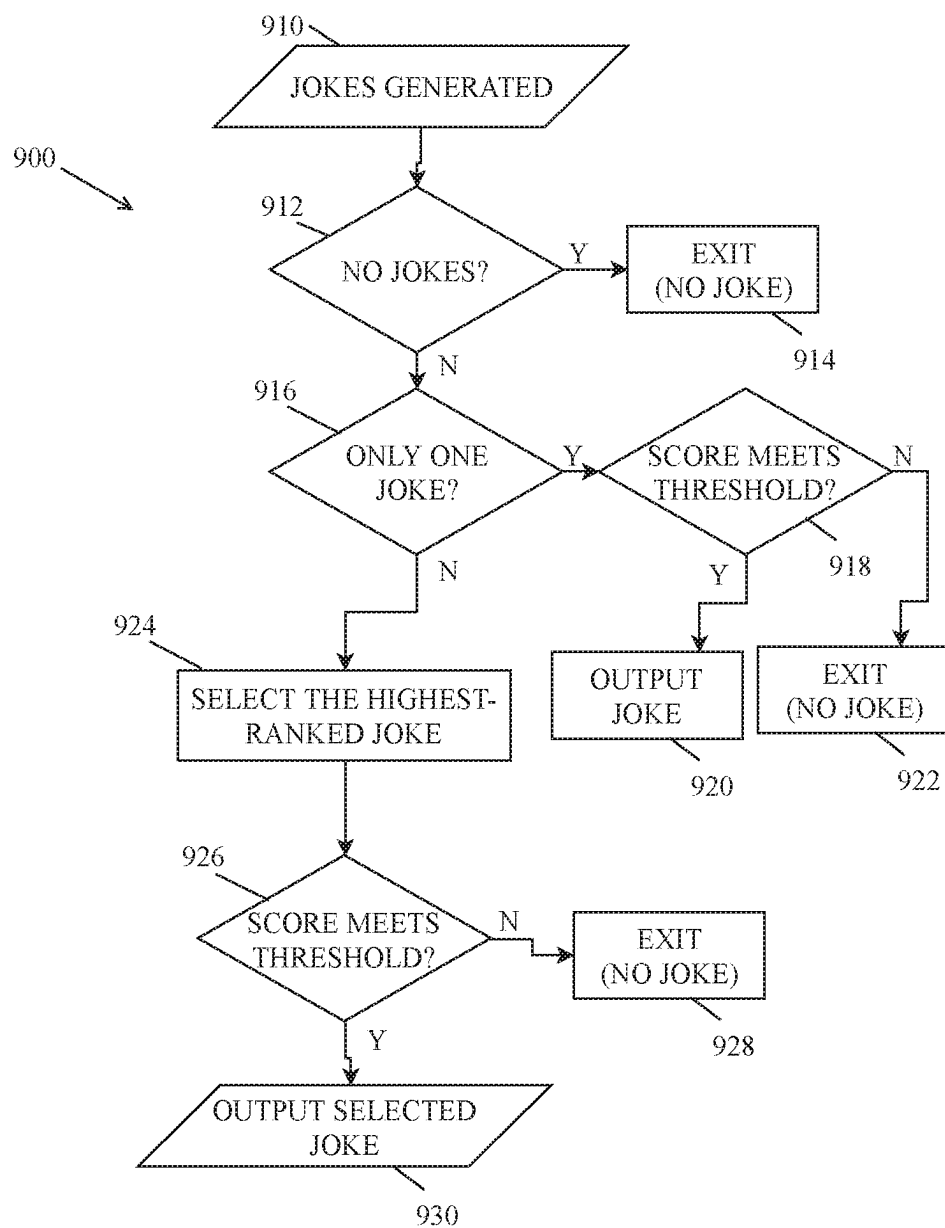


FIG. 9

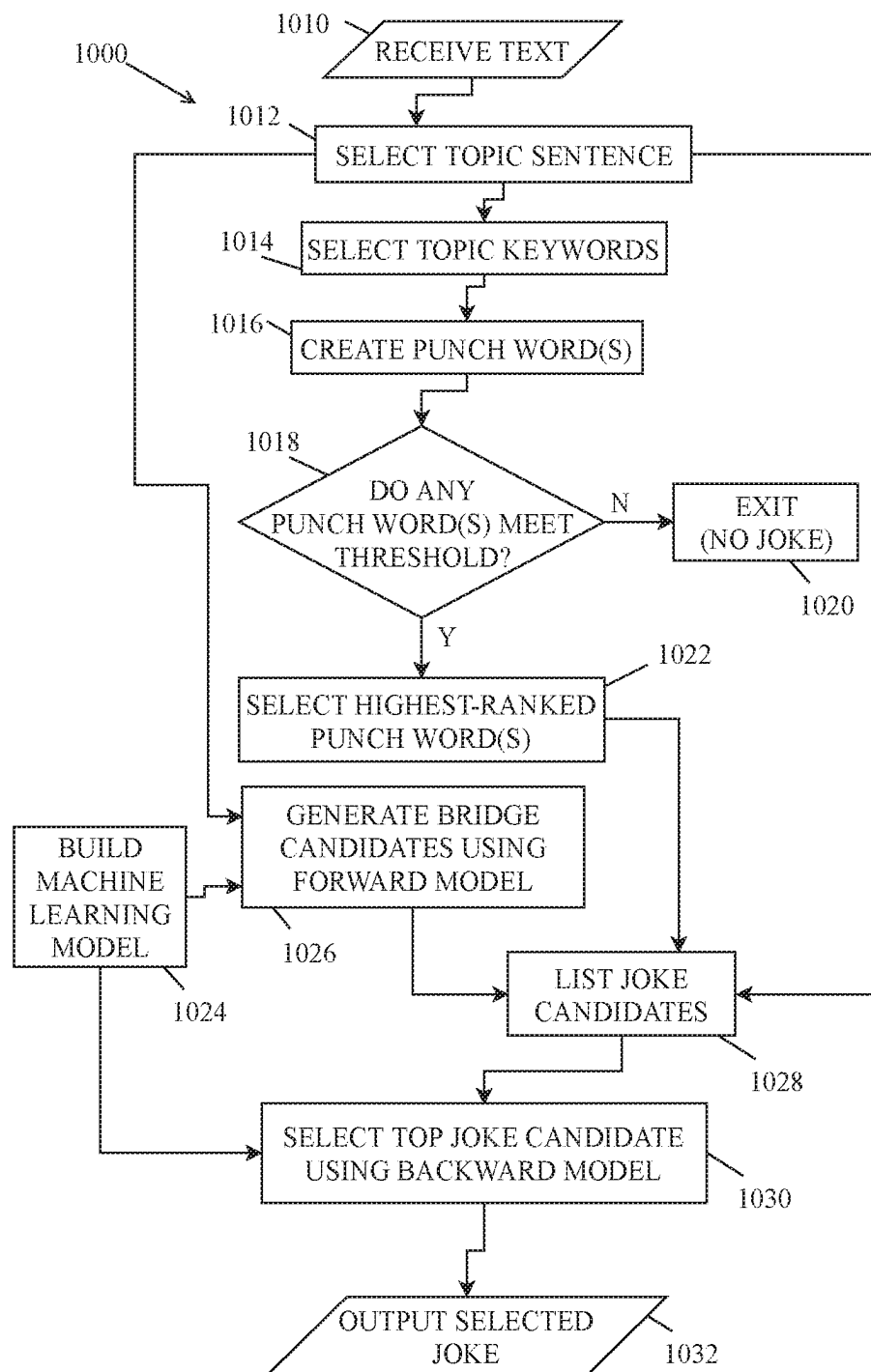


FIG. 10

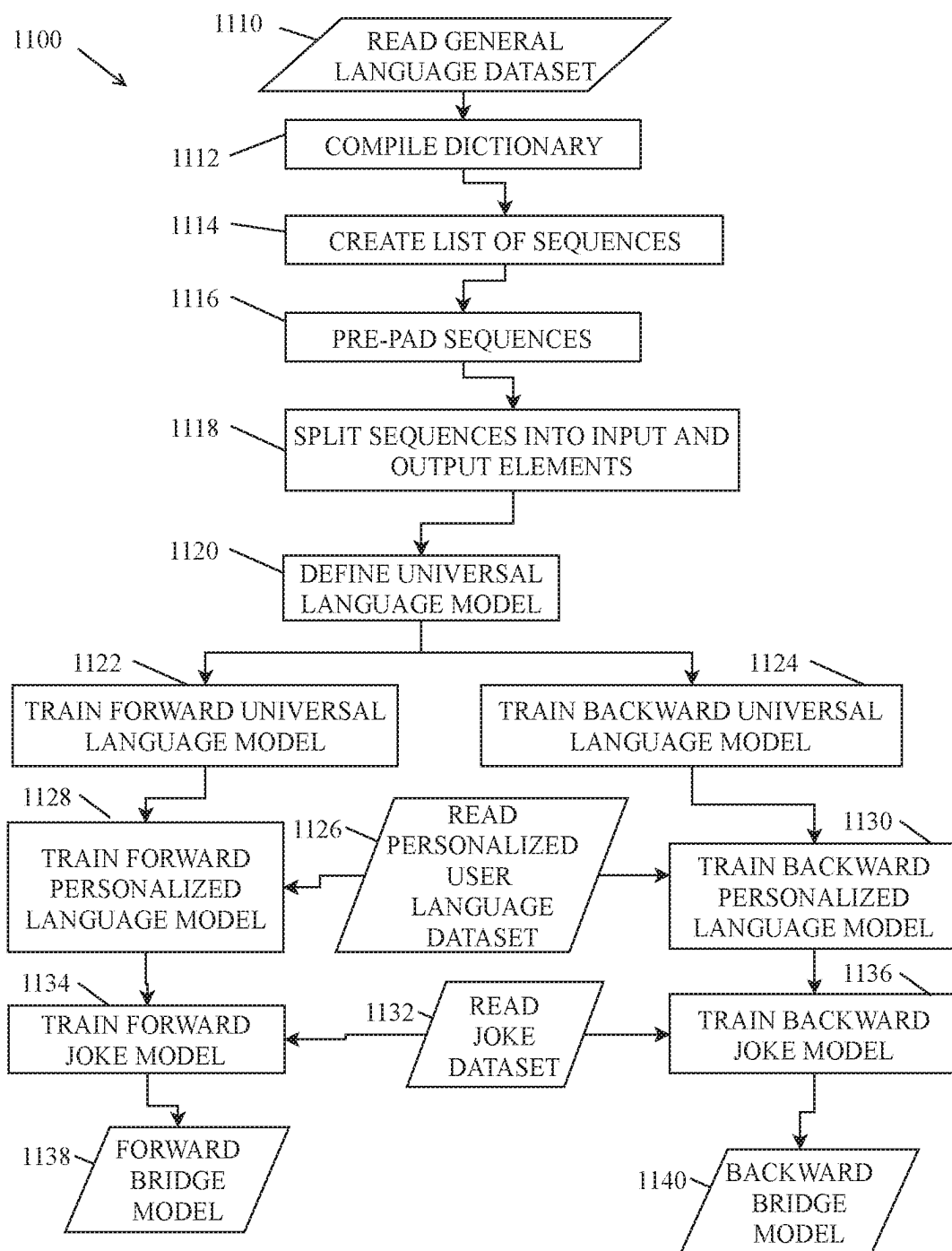


FIG. 11

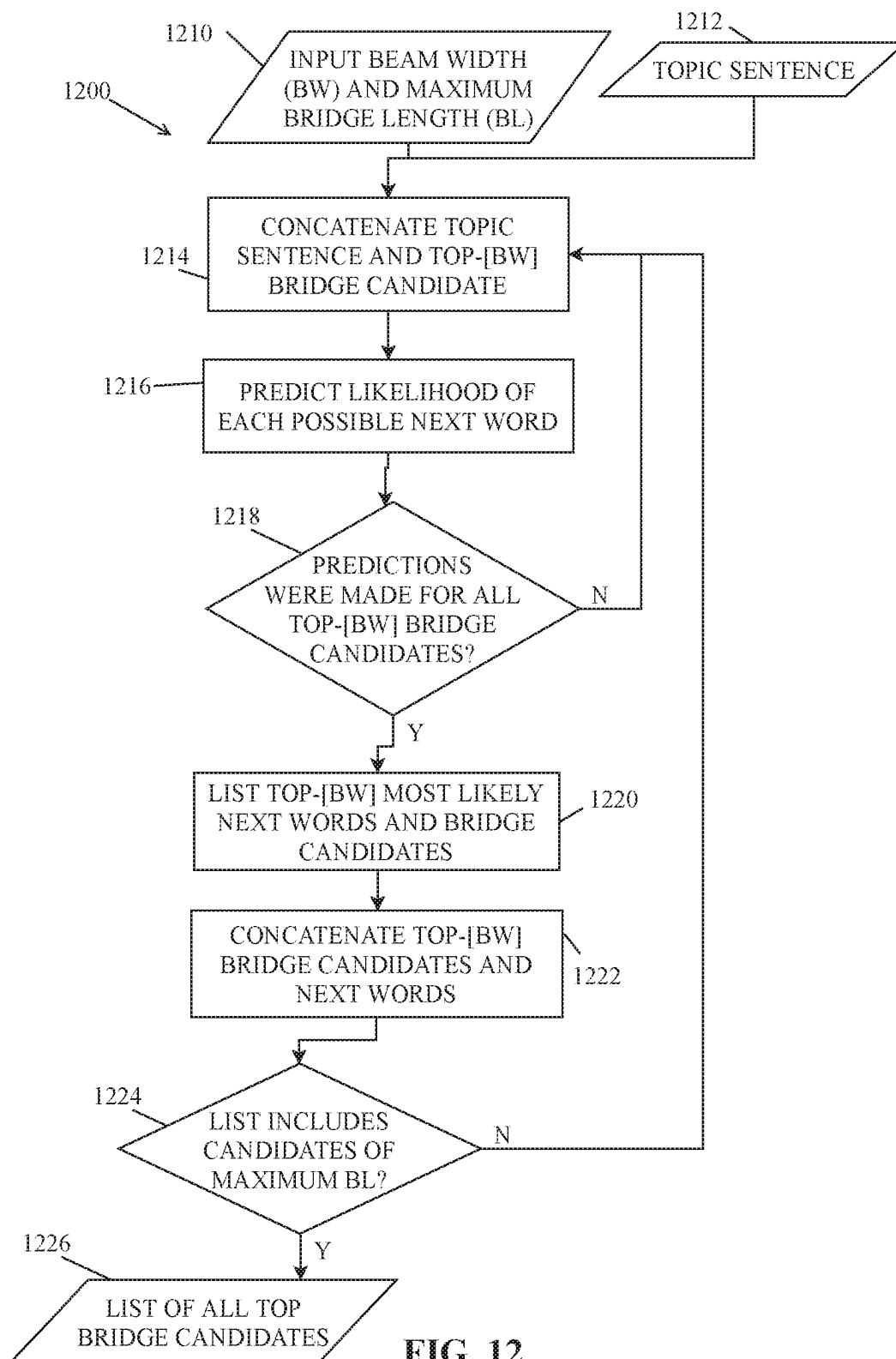


FIG. 12

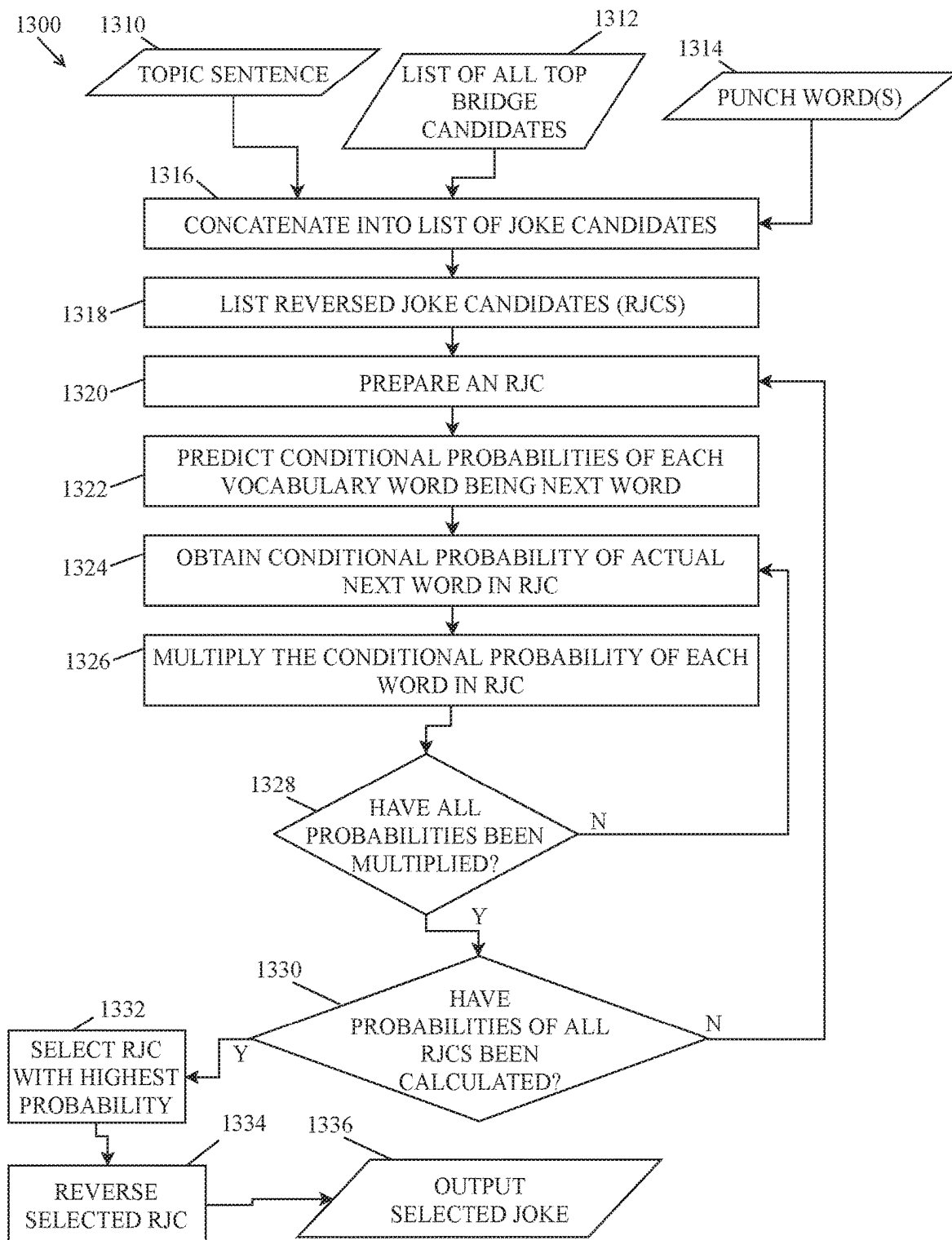
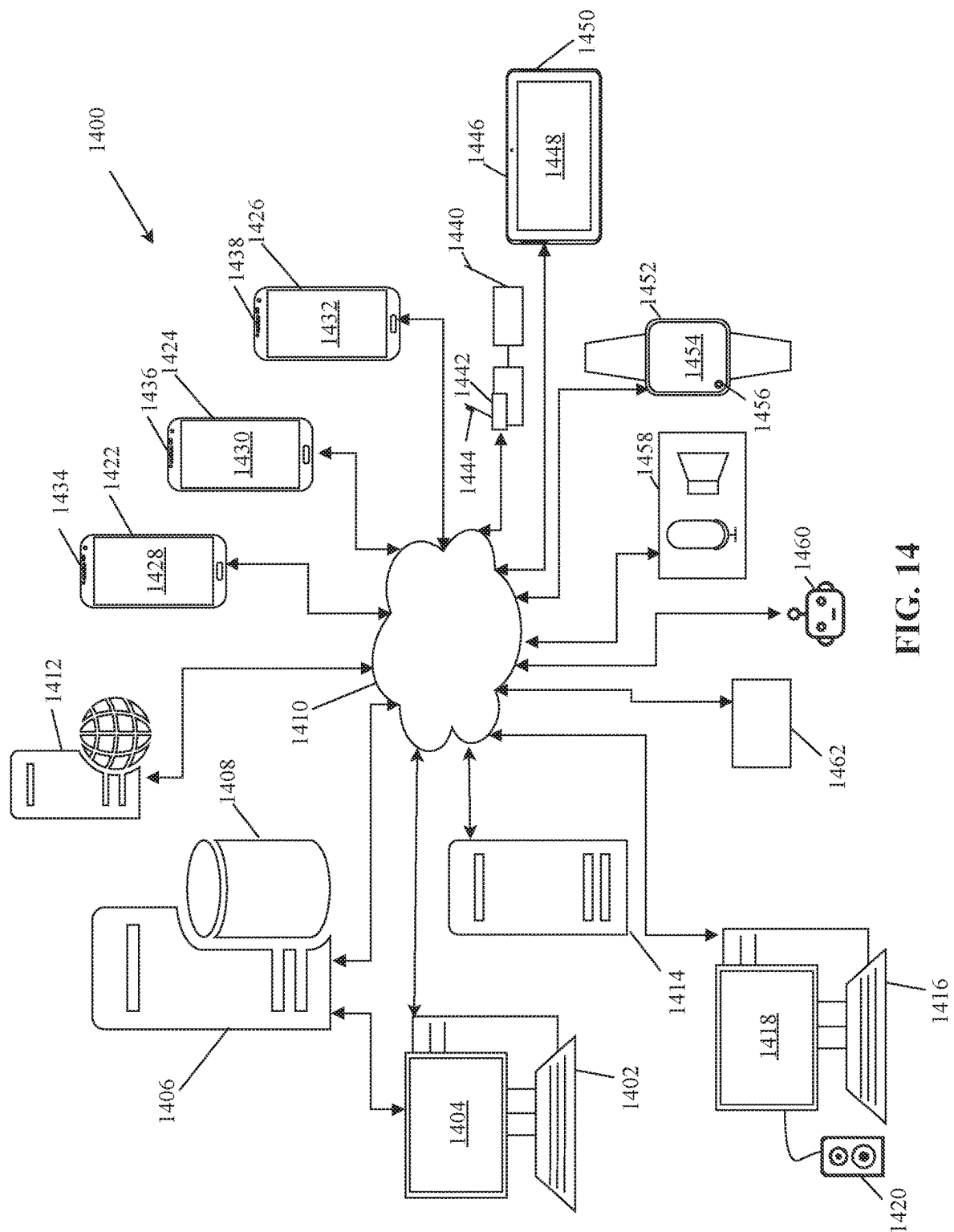
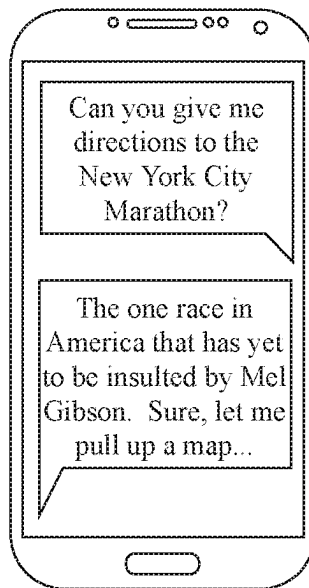


FIG. 13



**FIG. 15**

1

SYSTEMS AND METHODS FOR GENERATING COMEDY

CROSS REFERENCE TO RELATED APPLICATIONS

This document is a continuation-in-part of U.S. Nonprovisional patent application Ser. No. 16/250,972, entitled “Systems and Methods for Generating Jokes,” naming as first inventor Joseph E. Toplyn, which was filed on Jan. 17, 2019, which in turn claims the benefit of the filing date of U.S. Provisional Patent Application No. 62/634,845, entitled “System and Method for Generating Jokes,” naming as first inventor Joseph E. Toplyn, which was filed on Feb. 24, 2018, the disclosures of each of which are hereby incorporated entirely herein by reference.

BACKGROUND

1. Technical Field

Methods and systems disclosed herein relate generally to methods and systems for generating jokes and, more particularly, to methods and systems for generating original, contextually integrated jokes based on text that has been obtained from an external source.

2. Background Art

Non-computerized methods of creating monologue jokes having three components, i.e., topic, angle or bridge words, and punch line, exist in the art as discussed in, for example, Joe Toplyn, “Comedy Writing for Late-Night TV,” published by Twenty Lane Media, LLC (2014) (for example pages 53-57, 68-79, 85-87, and 98-111 of that publication discuss these elements).

Computational humor is a relatively new branch of computational linguistics and artificial intelligence (AI). One computational humor challenge that researchers have attempted to address is using a computer to generate jokes in a more humanlike, engaging, and effective fashion.

For example, the JAPE (Joke Analysis and Production Engine) program generates punning riddles. STANDUP (System To Augment Non-speakers’ Dialogue Using Puns) was developed as an extension to JAPE. The HAHA (Humorous Agents for Humorous Acronyms) acronym generator creates witty acronyms. The DEVIANT (Double Entendre via Noun Transfer) program finds sentences that can be turned into jokes by adding the sentence “That’s what she said.” But those joke generation programs are limited to certain joke formulas and formats.

The developers of AI virtual assistants like AMAZON’s ALEXA, APPLE’s SIRI, MICROSOFT’s CORTANA, and GOOGLE ASSISTANT sidestep the challenge of generating original jokes. Instead of ad-libbing newly created, contextually integrated jokes during their interactions with humans, those AI virtual assistants merely deliver, on request, jokes prewritten by humans that have been stored in a database.

SUMMARY

Embodiments of systems for generating jokes may include: one or more databases; one or more servers communicatively coupled with one or more databases; and a first computing device communicatively coupled with at least one of the one or more servers through a telecommunica-

2

tions network, wherein the one or more servers are configured to receive text from an external source communicatively coupled with the one or more servers through the telecommunications network and, in response, select one or more topic keywords, generate one or more punch words using words stored in the one or more databases which are related to the one or more topic keywords, use the one or more punch words to generate one or more jokes, and communicate a signal to the first computing device; and wherein the first computing device is configured to, in response to receiving the signal, display or speak one of the one or more jokes.

Embodiments of systems for generating jokes may include one or more or all of the following:

The external source may be the first computing device.

The received text may be text typed or spoken to a virtual assistant using the first computing device.

The external source may be a third party server and not the first computing device.

The one or more servers may be further configured to select a topic sentence from the received text.

The one or more servers may be further configured to add one or more bridges to the one or more punch words to generate the one or more jokes.

The one or more servers may be further configured to determine a highest ranking joke from among the one or more jokes.

Embodiments of methods for generating jokes may include: providing one or more servers communicatively coupled with one or more databases; in response to receiving text, at the one or more servers through a telecommunications network or at a first computing device: selecting one or more topic keywords from the received text using the one or more servers or using the first computing device; generating one or more punch words with the one or more servers using words stored in the one or more databases which are related to the one or more topic keywords; generating one or more jokes with the one or more servers using the one or more punch words; communicating a signal to the first computing device through the telecommunications network using the one or more servers; and in response to receiving the signal at the first computing device, displaying or speaking one of the one or more jokes using the first computing device.

Embodiments of methods for generating jokes may include one or more or all of the following:

The received text may be received at the one or more servers from the first computing device through the telecommunications network.

The received text may be text typed or spoken to a virtual assistant using the first computing device.

The received text may be received at the one or more servers from a third party server and not from the first computing device.

The method may further include selecting a topic sentence from the received text using the one or more servers or the first computing device.

The method may further include adding one or more bridges to the one or more punch words, using the one or more servers, to generate the one or more jokes.

The method may further include, using the one or more servers, determining a highest ranking joke from among the one or more jokes.

Embodiments of methods for generating jokes may include: providing one or more servers communicatively coupled with one or more databases; receiving text at the one or more servers from an external source communicatively coupled with the one or more servers through a telecommunica-

munications network; in response to receiving the text at the one or more servers; selecting a topic sentence from the received text using the one or more servers; selecting one or more topic keywords of the topic sentence using the one or more servers; generating one or more punch words with the one or more servers using words stored in the one or more databases which are related to the one or more topic keywords; adding one or more bridges to the one or more punch words, using the one or more servers, to generate one or more jokes; communicating a signal to a first computing device through the telecommunications network using the one or more servers; and in response to receiving the signal at the first computing device, displaying or speaking one of the one or more jokes using the first computing device.

Embodiments of methods for generating jokes may include one or more or all of the following:

Generating the one or more punch words may further include selecting a group of words stored in the one or more databases which are related to the one or more topic keywords, sorting the group of words into pairs, and calculating a wordplay score for each pair.

The method may further include, using the one or more servers, determining a highest ranking joke from among the one or more jokes, and further may include, in response to receiving the signal at the first computing device, displaying or speaking the highest ranking joke using the first computing device.

The one or more topic keywords may include a first topic keyword and a second topic keyword, and the method may further include, using the one or more servers, determining whether the first topic keyword is a string consisting of two or more tokens and, if so, calculating a wordplay score for each pairing of at least one of the two or more tokens with one of a plurality of words related to the second topic keyword.

Generating the one or more punch words may further include generating a portmanteau word using at least a portion of one of the words related to the one or more topic keywords.

The one or more topic keywords may include a first topic keyword and a second topic keyword, and generating the one or more punch words may include selecting a word chunk which is related to the first topic keyword and replacing one or more words of the word chunk with one or more words that are related to the second topic keyword.

Embodiments of methods for generating jokes may include: providing an electronic device including a processor and memory storing instructions for execution by the processor; accepting a text string from a text input or from a speech input received from a user or obtained from documents; extracting a sentence for a joke topic from a text string based on features such as whether the sentence is short enough and where the sentence falls in the text string; via the processor, analyzing the joke topic and identifying nouns, noun phrase chunks, named entities, and other words and phrases that are potentially the most useful for generating jokes; eliminating from consideration of potential use in generating jokes any of a plurality of text units in a list of stop words stored in a database and text units that are too similar to each other; determining, via the processor, which pair of words or phrases in the topic sentence is least likely to share the same context in a large, relevant text corpus and selecting those as the topic keywords to be used for generating jokes; determining, via the processor, a plurality of words related to the topic keywords in that they are semantically associated with the topic keywords in the context of a large, relevant, text corpus; scoring wordplay based on a

set of scoring rules that take into account features of a pair of words including but not limited to edit distance based on pronunciation, edit distance based on spelling, number of stop consonants based on spelling, number of stop consonants based on pronunciation, alliteration, rhyming, stemming, and lemmatization; via the processor, pairing words related to the topic keywords with each other and with the topic keywords themselves and determining which word pairs have the best wordplay so as to create punch word(s) with which to end the joke; to create a joke, adding bridge words selected at random from a preset list in a database to the topic keywords and the punch word(s).

Embodiments of methods for generating jokes may include one or more or all of the following:

Creating punch word(s) for a second type of joke by using wordplay to pair words related to one topic keyword with a token of a second topic keyword that consists of more than one token, or by using wordplay to pair words related to one topic keyword with words related to a token of a second topic keyword that consists of more than one token.

Creating punch word(s) for a third type of joke by exploiting wordplay to form a portmanteau word out of a word related to one topic keyword and a word related to a second topic keyword.

Creating punch word(s) for a fourth type of joke by substituting a word related to one topic keyword for one or more words with which it has adequate wordplay that are contained in a word chunk related to a second topic keyword.

To create a joke, adding to the topic and the punch word(s) bridge words that incorporate a definition, selected from a database, that pertains to one of the punch word(s).

To create a joke, starting with a topic, then creating punch word(s) based on that topic, and then adding bridge words that link that topic to the punch word(s) in such a way that the topic, bridge words, and punch word(s) taken together constitute a natural-sounding unit of text.

To create a joke that may strike a user as humorous, arranging the parts of the joke in this order: topic, bridge words, punch word(s).

To create a joke, adding to the topic sentence and the punch word(s) bridge words that have been generated using machine learning language models that make predictions based on the topic sentence and the punch word(s).

To create a joke using machine learning models, using a language model predicting forward from the topic, or a language model predicting backward from the punch word(s), or both.

To create a joke that may strike a user as more humorous, training the machine learning language models used in the joke generation process on personal language data that has been obtained from sources relating to that particular user.

To create a joke that may strike a user as more humorous, training the machine learning language models used in the joke generation process on a corpus of professionally written, monologue-style jokes.

Ranking jokes by the probability that they would occur in the context of a large corpus of user-relevant text, with the joke of highest probability considered to be the best joke.

Ranking jokes by their wordplay score and determining whether the wordplay score for a joke meets a preset threshold that indicates sufficient confidence in the funniness of the joke.

Providing an output to the user, wherein the output is a joke that is original, humorous, natural-sounding, and integrated into the context of the input received by the system.

5

To create a joke based on a topic, linking in a surprising way two topic keywords and words related to them by using reasoning informed by commonsense knowledge and informed by the anticipated world knowledge, beliefs, and attitudes of the intended audience of the joke to be generated.

To create a joke based on a topic, utilizing expert-based rules and machine learning, either individually or in combination.

Implementations of jokes generated using systems and methods disclosed herein include three elements: a topic sentence or topic words; one or more bridge words or angles; and one or more punch words or punchlines. Other implementations of jokes generated using systems and methods disclosed herein include two elements generated in response to receiving a topic sentence: one or more bridge words or angles; and one or more punch words or punchlines (for example, in those implementations the topic may only be in the topic sentence that was received).

General details of the above-described embodiments, and other embodiments, are given below in the DESCRIPTION, the DRAWINGS, and the CLAIMS.

BRIEF DESCRIPTION OF THE DRAWINGS

Embodiments will be discussed hereafter using reference to the included drawings, briefly described below, wherein like designations refer to like elements:

FIG. 1 illustrates an example of a computing environment that may be used in the systems to implement the methods;

FIG. 2 illustrates a flowchart of an example of a method for generating jokes implemented using the system of FIG. 14;

FIG. 3 illustrates a flowchart of an example of a method of selecting a topic sentence using the system of FIG. 14;

FIG. 4 illustrates a flowchart of an example of a method for selecting topic keywords using the system of FIG. 14;

FIG. 5 illustrates a flowchart of an example of a method for generating a joke of a first type (Type #1) using the system of FIG. 14;

FIG. 6 illustrates a flowchart of an example of a method for calculating a score for the wordplay between two words or phrases using the system of FIG. 14;

FIG. 7 illustrates a flowchart of an example of a method for generating a joke of a second type (Type #2) using the system of FIG. 14;

FIG. 8 illustrates a flowchart of an example of a method used to generate a joke of a third type (Type #3) and a joke of a fourth type (Type #4) using the system of FIG. 14;

FIG. 9 illustrates a flowchart of a method of ranking and outputting one of multiple generated jokes using the system of FIG. 14;

FIG. 10 illustrates a flowchart of an example of a machine learning method used for generating jokes using the system of FIG. 14;

FIG. 11 illustrates a flowchart of an example of a method of building machine learning language models for use in joke generation methods using the system of FIG. 14;

FIG. 12 illustrates a flowchart of an example of a method of using machine learning language models to generate bridge candidates using the system of FIG. 14;

FIG. 13 illustrates a flowchart of an example of a method of outputting a best joke generated with machine learning using the system of FIG. 14;

FIG. 14 illustrates a representative example of a system for generating jokes; and

6

FIG. 15 illustrates an example of an interface of an end user computing device displaying a joke delivered by one or more elements of the system of FIG. 14.

DESCRIPTION

Implementations/embodiments disclosed herein (including those not expressly discussed in detail) are not limited to the particular components or procedures described herein. Additional or alternative components, assembly procedures, and/or methods of use consistent with the intended systems and methods for generating jokes may be utilized in any implementation. This may include any materials, components, sub-components, methods, sub-methods, steps, and so forth.

It is an object of the present invention to provide novel joke generating systems and methods free from the limitations inherent in the above-described conventional joke generation systems. Various of the disclosed embodiments concern systems, methods, and computer-readable storage media for responding to input obtained from a user via an interactive device, or from a database, by generating an original, contextually integrated (relevant), and humorous joke that requires no human editing. In some embodiments, a user may interact with the system via a mobile device, tablet, desktop computer, laptop computer, robot (including social robot), doll, toy, game, telephone, audio/video interface, chatbot, digital assistant, voice assistant, AI virtual assistant, conversational agent, smart speaker, virtual reality (VR) interface, virtual agent, embodied virtual agent, embodied avatar, smart home appliance, smart home device, smart environment, smart furniture, voicebot, chatbot, or the like. During the interaction, the system may receive a text string as an input. In some implementations this may involve software of the system converting a user's audio input (spoken sentence, paragraph, etc.) into a text string and inputting that text string into a component or element of the system. A topic sentence is selected from the text string and keywords in the topic sentence are identified and extracted. For each keyword, related words or phrases previously stored in a database of the system are identified and various features are extracted from them. The most promising related words or phrases are linked in various ways to form candidate punch words. The system ranks the candidate punch words and then selects the punch words that rank the highest and that also meet a preset threshold. To those punch words and the topic sentence, the system adds bridge words that are related to the topic sentence and to the punch words, thus creating an original, contextually integrated, and humorous joke response. The joke response is then output to the user in audio and/or visual format.

Disclosed embodiments include methods and systems which receive and analyze textual data and respond with textual jokes. Textual data may include written or audio text. The textual data can be obtained, for example, from web pages, social media posts, or other documents available over a global computer network such as the Internet. The textual data can also be obtained from a speech-to-text converter which recognizes speech such as words spoken by a user to an AI virtual assistant, chatbot, robot, smart speaker, and the like and transforms it into text.

The methods and systems can be implemented using a variety of computing environments and devices, including personal computers, server computers, mobile devices, smart speakers, database servers, databases, web servers, smart watches, smart glasses, tablets, laptops, a telecommunications network (such as the Internet), etc. Before describ-

ing the embodiments in detail, a discussion of example computing environments in which the embodiments can be implemented may be useful. FIG. 1 depicts a high-level block diagram of a general-purpose computing device suitable for use in performing some of the functions described herein. As depicted in FIG. 1, the computing device 100 comprises a processor 110 (e.g. a CPU); memory 112, e.g. random-access memory (RAM) and/or read-only memory (ROM); software 114 for implementing the method (or portions of the method) described herein for generating one or more jokes; and various additional components 116 which may be implemented as hardware and/or software (such as, by non-limiting example, a receiver and transmitter for communicating with other computing devices of the system locally and/or through a telecommunications network, wireless communication elements, a speaker for delivering generated jokes in audio format, a display for delivering generated jokes in visual format, a text-to-speech synthesizer for converting text to speech, a speech-to-text converter, user input elements such as a keyboard, keypad, touchscreen, mouse, microphone, and the like). The double-arrowed connectors of FIG. 1 representatively illustrate the individual elements being able to communicate with one another.

It should be noted that portions of the system and method can be implemented using software and/or a combination of software and hardware, e.g. using application specific integrated circuits (ASIC), general purpose computers, or any other hardware equivalents. In one embodiment, the method may be implemented using software 114 for generating one or more jokes, the software loaded into memory 112 and executed by processor 110 to implement the functions as discussed above. As such, in implementations the method for generating jokes (including associated data structures) can be implemented using a non-transitory computer-readable storage medium, e.g. RAM memory, magnetic or optical drive or diskette, and the like.

Referring now to FIG. 14, a more detailed example of a system 1400 for implementing joke generation methods (system for generating jokes) (system) is shown. The system includes a plurality of computing devices, each of which may include some or all of the elements described previously with respect to FIG. 1.

For example, computing device (device) 1402 includes interface 1404 and is an administrator device. This device may be used by an administrator (admin) to populate a database 1408, using database server (DB server) 1406, with some of the elements of the system as will be described hereafter, and to make changes to the database elements. In the implementation shown the admin device is coupled with the DB server directly, though also through a telecommunications network 1410 which may be, by non-limiting example, the Internet, though it could also be a local area network (LAN) in a more localized deployment of the system. The admin device could also communicatively couple with the database/DB server through a web server 1412, such as by accessing a website using credentials. The admin device is shown as a desktop computer but could be a laptop, mobile phone, tablet, smart watch, smart speaker, smart glasses, and so forth.

In implementations the admin computer, DB server, database, web server, and the like could all be implemented on a single machine (such as using virtual servers on a single machine) but in large network deployments there will more likely be a plurality of cloud-connected server racks used to implement these various elements as the number of users scales up. For example, AMAZON WEB SERVICES

(AWS) server racks may be utilized to implement the database server, the database, the web server, and so forth so that the number of users may be increased to very large numbers.

In implementations the system may interact with third party server(s)/database(s) 1414 to gather information. As has been described to some extent above, for example, the system may scour the Internet for contextual information which will be used in joke generation. This may involve processing online news stories, online encyclopedia articles, online social media content, and so forth (all of which will be stored on third party servers/databases) and storing portions of the same in database 1408 to be used in joke generation.

System 1400 shows a number of user devices simultaneously communicatively coupled with the database 1408 via the telecommunications network, including a personal computer (computer) 1416 having a display 1418 and an associated microphone/speaker 1420, a number of mobile phones 1422-1426 each having a display (1428-1432) and microphone/speaker elements (1434-1438), smart glasses 1440 including a display 1442 and microphone/speaker elements 1444 (a representative example of this would be GOOGLE GLASS), a mobile tablet (tablet) 1446 including display 1448 and microphone/speaker elements 1450, a smart watch 1452 including a display 1454 and microphone/speaker elements 1456, a smart speaker 1458 including microphone/speaker elements (representative examples would be GOOGLE HOME, AMAZON ECHO, APPLE HOMEPOD, and the like), a robot 1460 (which may be a humanoid robot, social robot, talking head, avatar, embodied virtual agent, embodied avatar, or otherwise and which may include microphone/speaker elements for user interaction), and any other human interaction device (device) 1462 which may include microphone/speaker and other input/output elements for user interaction (by non-limiting example, user interfaces integrated in automobiles, trucks, buses, airplanes, telephones, interactive advertising displays, street furniture, Internet of Things devices, wearable technology, virtual receptionist kiosks, and any other human interaction device now known or hereafter discovered/invented).

The system at any given moment may have fewer or more user devices communicatively coupled with it, and each type of device may be scaled up to any number of users, so that FIG. 14 is a simplified diagram. In implementations, for example, there may be thousands of users or more simultaneously interacting with the system for joke generation, and the system and methods herein may interact with third party software/elements on the user devices (which may involve their own accessing of remote servers and databases and the like). For example, GOOGLE ASSISTANT, AMAZON's ALEXA, or APPLE's SIRI may interact with the system so that the jokes generated are then relayed to the user through those elements. The system may also have other elements for implementing the method(s), such as one or more application servers communicatively coupled with one or more other elements of the system.

While various embodiments of joke generating systems have been described above at a high level, these have been presented by way of example only, and not limitation. Thus, the breadth and scope of a preferred embodiment of a joke generating system should not be limited by any of the above-described exemplary embodiments.

Following are joke generation methods which can be implemented using joke generating systems such as those shown in FIGS. 1 and 14.

FIG. 2 illustrates an example high-level flowchart of a method 200 for generating jokes, which jokes may be original, contextually integrated jokes based on text that has been obtained from a source external to the server(s) of the system. In this exemplary embodiment, the jokes to be generated comprise three parts, in this order: A) the topic sentence, which is received from a source external to the server(s) of system 1400 (for example from end user devices 1416, 1422, 1424, 1426, 1440, 1446, 1452, 1458, 1460, 1462, and/or from one or more news websites and/or one or more other websites); B) bridge words, which smoothly connect the topic sentence to the punch word or punch words in a way that results in a joke consisting of a natural-sounding sentence or sentences (generated/supplied by one or more of the servers of the system in conjunction with the one or more databases 1408); and C) a punch word or punch words, which are intended to surprise the user by introducing an incongruity into the joke that suddenly turns out to be harmless and thereby to cause the user to laugh (generated/supplied by one or more of the servers of the system in conjunction with the one or more databases 1408). At step 210, one or more servers of the system receive text from an external text source, by non-limiting examples from a website or from a human interaction device (such as by non-limiting examples elements 1416, 1422, 1424, 1426, 1440, 1446, 1452, 1458, 1460, 1462, one or more news or other websites (such as by scraping websites), etc.). At step 212, the method selects from that text a topic sentence from which the joke generation process will begin. From that topic sentence, a pair of topic keywords is selected in step 214; in an alternative embodiment, only one keyword may be selected. From those topic keywords, a punch word or punch words intended to trigger a laugh from the user in the context of the joke are created in step 216. In step 218, the punch word(s) are linked together with the topic and with bridge words to form candidates for jokes; the bridge words are words that the method generates which smoothly connect the topic sentence to the punch word(s) to form a joke that consists of a natural-sounding sentence or sentences. The bridge words may in implementations be called the “angle” and may include characteristics and/or elements of angles disclosed in Joe Toplyn, “Comedy Writing for Late-Night TV,” published by Twenty Lane Media, LLC (2014), the entirety of which is hereby incorporated herein by reference (for example pages 68-70 of that publication discuss angles). The joke candidates that have been generated are ranked in order of their humor potential in step 220 and the highest-ranking joke is determined. Finally, if the highest-ranking joke meets a preset threshold, it is output to the user in step 222, such as by being displayed visually or output in audio format using interfaces of devices 1416, 1422, 1424, 1426, 1440, 1446, 1452, 1458, 1460, 1462. To illustrate in more detail the method broadly outlined in FIG. 2, detailed examples of block diagrams that describe the various steps in the method follow.

Topic Sentence Selection

Referring now to FIG. 3, shown is a block diagram of a method 300 to select a topic sentence from which the joke generation process will begin. This method may be implemented, by non-limiting example, using software of the database server and/or software installed on the end user device. In an exemplary embodiment, this method attempts to obtain input text at step 312, for example, from text source 310. In an exemplary embodiment, text source 310 may be the output of a speech-to-text converter such as GOOGLE CLOUD SPEECH-TO-TEXT, which would be the case if, in an exemplary embodiment, the user were speaking to one of

the user interaction devices shown in FIG. 14, though it could also be text typed into one of the interaction devices (by non-limiting example, devices 1416, 1422, 1424, 1426, 1440, 1446, 1452, 1458, 1460, 1462) or obtained from one or more news or other websites, etc. If at decision step 314 it is determined that no text has been input, then the method may attempt again to obtain input text.

Once input text has been obtained, the sentences in the text are listed at step 316 and the first sentence is extracted. The first sentence of a text string may be the most likely candidate to be the topic of a joke because it may be most likely to be the most attention-getting sentence in the text string.

Decision step 318 determines whether the number of words in the first sentence of the input text exceeds a preset threshold, for example, 20 words (or some other threshold input by the admin using the admin computer and, for example, stored in the database). A long sentence may be an unlikely candidate to be the topic of a joke because it likely contains too many distracting elements that can divide the attention of the joke’s intended audience. If at decision step 318 it is determined that the first sentence is too long, then the method may attempt to obtain new input text. If at decision step 318 the first sentence is determined to be of acceptable length, then at step 320 that sentence is output as the topic sentence from which joke generation will be attempted. If the first sentence is too long, in implementations the method may include reviewing the next listed sentences, in order, until one at or below the threshold is found.

By non-limiting example, and referring back to FIG. 14, any individual user device (mobile phone, tablet, etc.) may have software installed on it that performs some of these functions, such as selecting the topic sentence using the above method, and then the topic sentence may be communicated to the database server (or another system-coupled server) for other method steps to be performed, such as the remainder of the joke generation steps, and then the joke is communicated from the database server to the user device using the telecommunications network. In other implementations, the user device may communicate all of the sentences to the database server (or another system-coupled server) and the server may perform the step of selecting the topic sentence and other method steps to generate the joke and send the joke to the user device. In other implementations the database server or other system-coupled servers may perform some functions of the joke generation but the remainder of the joke generation may be done at the user device end, for example the database server or other server (s) sending certain elements of a joke to the user device and software installed on the user device then incorporating those elements of the joke to form a completed joke for communication to the user. In other implementations all of the steps may be performed by the user device using software installed on the user device and using locally-stored elements instead of accessing a remote database. In short, the steps of joke generation described herein may be divided up between software installed on user devices and other elements of the system of FIG. 14 in any manner to best accomplish the method. Nevertheless, implementations having a larger overall system such as in FIG. 14 may allow for much larger libraries of information stored in the database for the system to be able to create contextualized jokes (such as jokes based on recent news) and may allow the joke generation methods to be implemented simultaneously for very large numbers of users. Furthermore, software at the user-device end or at servers or other elements of the system

may be used to implement the various logic steps/trees displayed in the Figures, as will be understood by the practitioner of ordinary skill in the art.

Any of a variety of known natural language processing packages can be used to extract the first sentence and count the number of words in it. In one exemplary embodiment, the NATURAL LANGUAGE TOOLKIT (NLTK) developed by Steven Bird and Edward Loper of the University of Pennsylvania has sentence and word tokenizers that can accomplish the tasks.

Topic Keyword Selection

Once a topic sentence has been selected and output as in FIG. 3, topic keywords are extracted from it. FIG. 4 shows an exemplary embodiment of a method 400 for extracting topic keywords. Again, some or all of these steps may be performed at the user device level using installed software and other aspects may be performed using the DB server and/or other servers of the system.

One example set of topic keyword candidates consists of nouns, noun phrase chunks, and named entities. Nouns are self-explanatory. Noun phrase chunks are multi-word sequences that correspond to individual noun phrases. For example, "long-running reality show" is a noun phrase chunk. Named entities are definite noun phrases that refer to specific types of entities such as organizations, persons, locations, dates, times, amounts of money, percentages, facilities, and geopolitical entities. For example, "Mount Everest" is a named entity.

Nouns, noun phrase chunks, and named entities are the most likely candidates to be useful topic keywords because they are likely to be the most attention-getting words or phrases in the topic sentence. And the most attention-getting words or phrases in the topic sentence are the elements of the topic sentence that are most likely to lead to the generation of a funny joke. Other embodiments of this system may also include verbs, verb phrase chunks, and adjectives in the set of candidates for topic keywords.

Word-level tokenization, part-of-speech (POS) tagging, chunking, and named entity recognition can be accomplished with any of a number of natural language processing tools. One exemplary embodiment will use the tools provided in the NLTK.

Turning to FIG. 4, block 410 represents the topic sentence that was output in step 320. In step 412, the named entities in the topic sentence are extracted using the tools of word-level tokenization, POS tagging, chunking, and named entity recognition, or the like. At decision step 414 a determination is made as to whether any of a set of excluding rules apply to the list of extracted named entities. One example of an excluding rule involves the number of letters in the named entity. If a named entity is shorter than two letters, it is excluded in step 416. Another example of an excluding rule involves the presence of the named entity or its lower-case version on a list of stop words stored in a database. Any named entity on the list of stop words is excluded in step 416. Any named entity that is not excluded by the excluding rules is added, in step 418, to the list of candidates for topic keywords. The excluding rules may in implementations be set by an admin using the admin computer. Also, the system may couple with third-party databases and use third-party excluding rules, and in general the system may include more servers, database servers, application servers, and the like. For example some of the functionalities described herein (such as selecting topic sentences, applying excluding rules, and other steps described below) could be handled by one or more application servers of the system before querying the database using the database server to obtain elements to

generate a joke, which joke may be generated by the application server and merely communicated to the user device.

Referring back to stop words, the stop words used by the method include the most common words in the English language and other words that are unlikely to be useful as topic keywords from which to generate a funny joke. An exemplary, but not exhaustive, list of some stop words is as follows: afternoon; anybody; both; company; dozen; every-one; four; guy; hello; hundred; itself; look; Monday; nowhere; outside; regards; self; sort; thanks; thing; week; whatever; woman; yourself; and so forth. The stop words may be chosen by the admin using the admin computer and/or may be chosen using static or dynamic third-party lists of stop words accessed by elements of the system.

In step 420, the nouns in the topic sentence are extracted using the tools of word-level tokenization and POS tagging, or the like. Singular and plural nouns are extracted, including singular and plural proper nouns. At decision step 422 a determination is made as to whether any of a set of excluding rules apply to the list of extracted nouns. Examples of excluding rules are rules that exclude a noun if it is shorter than two letters or if it or its lower-case version is on the list of stop words used by the method. Nouns are also excluded if they are not alphanumeric but are included if they start with "\$." Another reason to exclude a noun is that the noun is present as a separate word in one of the named entities extracted from the topic sentence. To apply this excluding rule, the named entities extracted from the topic sentence have first been split into separate words. Step 424 excludes any noun excluded by the excluding rules. Any noun that is not excluded by the excluding rules is added, in step 418, to the list of candidates for topic keywords.

In step 426, the noun phrase chunks in the topic sentence are extracted using the tools of word-level tokenization and POS tagging, or the like. Also used is a chunk grammar consisting of rules that indicate how the topic sentence should be chunked. In one exemplary embodiment, the chunk grammar rules say that a noun phrase chunk should be formed whenever the chunker finds an optional adjective followed by one or two singular or plural nouns, excluding proper nouns. Another rule of the chunk grammar says that dollar amounts should be extracted as chunks. Using this chunk grammar, a chunk parser is created using, for example, the RegexpParser chunker in the NLTK. The chunk parser is used to extract noun phrase chunks, including dollar amounts, from the topic sentence.

At decision step 428 a determination is made as to whether any of a set of excluding rules apply to the list of extracted noun phrase chunks. Examples of excluding rules are rules that exclude a noun phrase chunk if it is shorter than two letters, if it or its lower-case version is on the list of stop words, or if it is present as a separate word or phrase in one of the named entities that were extracted from the topic sentence. Step 430 excludes any noun phrase chunks excluded by the excluding rules. Any noun phrase chunk that is not excluded by the excluding rules is added, in step 418, to the list of candidates for topic keywords and any duplicate candidates are removed from that list.

In implementations, to generate a joke based on the selected topic sentence, the most promising two keywords are chosen from the list of candidates for topic keywords. A funny joke may start with a topic sentence that gets the attention of its audience. So the most promising two keywords in the selected topic sentence may be the two most attention-getting keywords in that sentence. And the two most attention-getting keywords in the topic sentence may

be those two keywords which are least likely to appear together in a sentence, that is, the two keywords which an audience would consider to be the least likely to share the same context.

To measure how likely it is that two particular words will share a common context it is possible, in an exemplary embodiment, to measure how close their word vectors are in an appropriate vector space. This vector space can be produced by a machine learning model trained on a very large corpus of audience-relevant text in step 432. An exemplary embodiment might use GOOGLE's pretrained WORD2VEC model trained on approximately 100 billion words from a GOOGLE NEWS dataset; that model includes word vectors for a vocabulary of 3 million English words and phrases. Other embodiments might use machine learning models that were also trained on data scraped from online social media; pop culture websites; online forums, message boards, and discussion groups; and other data sources that represent the anticipated knowledge, beliefs, and attitudes of the intended audience for the joke or jokes to be generated. Other embodiments might use machine learning models that are even more tailored to an individual user because they have also been trained on that user's most frequently visited websites, websites in the user's online search and browsing history, social media posts "liked" by the user, the user's email, the user's communications with an AI virtual assistant, and other personal data sources.

In implementations the first step in selecting two topic keywords from the list of candidates for topic keywords is step 434, which filters out candidates that are not in the vocabulary of the machine learning model being used. Decision step 436 determines whether fewer than two candidates for topic keywords remain after the filtration process. In implementations, if fewer than two candidates remain then the method exits, as shown at step 438, and the method will not attempt to generate a joke. In other implementations, if fewer than two candidates for topic keywords remain, then the method may attempt again to obtain input text. If two or more candidates for topic keywords remain then the similarity, based on the machine learning model being used, of each and every pair of remaining candidates is calculated in step 440. An exemplary embodiment might use the "similarity" function of GOOGLE's pretrained WORD2VEC model trained on GOOGLE NEWS data to calculate the similarity of each pair of candidates. An exemplary embodiment would also filter out, in step 442, any pair of candidates with a calculated similarity less than zero; such pairs of candidates tend to be too dissimilar to be useful as topic keywords.

Whichever two remaining topic keyword candidates are the least similar, as indicated by their pairwise similarity calculation, become the two selected topic keywords in step 444 and are outputted in step 446. These are the two topic keywords that the method will use in attempting to generate jokes. Because those two topic keywords are the least similar pair, they may be the elements in the topic sentence that are most likely to attract an audience's attention. This means that a joke based on those two topic keywords may be most likely to strike the audience as funny. In implementations, if the two remaining topic keyword candidates that are the least similar, as indicated by their pairwise similarity calculation, have a calculated similarity that does not fall below a certain preset threshold, which in implementations may be set by the admin, then the method may not attempt to generate jokes. This is because a joke based on two topic keywords that are not sufficiently dissimilar may be unlikely to strike the audience as funny. Other embodiments of this

system may select as topic keywords those two keywords in the topic sentence that are computationally determined to convey the most information to the anticipated audience or are computationally determined to be the elements of the topic sentence that are likely to be the most distinctive, attention-getting, or surprising to the anticipated audience. If no two keywords in the topic sentence are computationally determined to be sufficiently informative, distinctive, attention-getting, or surprising to the anticipated audience, then the method may not attempt to generate jokes.

Generation of Joke of Type #1

In implementations, after two topic keywords have been selected as in FIG. 4, a joke of Type #1 is generated from them. FIG. 5 shows an exemplary embodiment of a method 500 of generating a first type of joke (Type #1). Block 510 represents the two topic keywords that were outputted in step 446. In step 512, the machine learning model trained in step 432 is used to list the words that are most similar to each topic keyword. That is, for each of the two topic keywords, a list is created of the top fifty, for example (which number in implementations may be selected by the admin), words that occur most frequently in contexts where the topic keyword appears. An exemplary embodiment might use the "most_similar" function of GOOGLE's pretrained WORD2VEC model trained on GOOGLE NEWS data to build the list of most similar words, which we'll refer to as related words. The topic keyword or one or more of its related words may consist of more than one token. The list of related words for a topic keyword may be filtered to exclude words that are poorly suited for use in assembling punch words, by non-limiting example, duplicate words, misspelled words, acronyms, abbreviations, words with non-alphabetic characters, or words unrelated to the sense of the topic keyword as it is used in the topic sentence. In other implementations, related words, associated words, or most similar words for a topic keyword, including words that occur most frequently in contexts where the topic keyword appears, may also be selected from a database or databases such as, by non-limiting example, a spreadsheet, knowledge base, graph database, relational database, semantic network, entity relationship model, dictionary, thesaurus, lexicon, or WORDNET (created by George A. Miller).

A separate list of related words is compiled for each topic keyword, i.e. for topic keyword A and for topic keyword B. Then each and every related word on the list for topic keyword A, including topic keyword A itself, is paired with each and every related word on the list for topic keyword B, including topic keyword B itself. The wordplay score of each pair of words is calculated in step 514. The process of calculating a wordplay score is described in detail in FIG. 6. Decision step 516 determines whether a pair of related words exists with a wordplay score that is better than a certain preset threshold (which may be determined by the admin in implementations). If such a word pair does not exist, then the method exits, as shown at step 518, and the method will not attempt to generate a joke of Type #1.

If there does exist a pair of related words with a wordplay score that is better than the preset threshold, then the pair of related words with the best wordplay score better than the threshold becomes the pair of punch words for a joke of Type #1 at step 520. The wordplay between the two punch words, because it is unexpected and surprising in the context of the rest of the joke, will result in the joke's humorous effect.

In step 522 the two punch words, punch word A and punch word B, are arranged in an optimal order that will make the joke read and sound as much as possible like natural English

speech. To do this, each of the two punch words is first tagged with its part of speech using the tools of word-level tokenization and POS tagging. If punch word A is any type of adjective, it is placed before punch word B in the joke, and its related topic keyword A is placed before topic keyword B. If punch word B is any type of adjective, it is placed before punch word A in the joke, and its related topic keyword B is placed before topic keyword A. If neither punch word is any type of adjective but punch word B begins with a capital letter, punch word B is placed before punch word A in the joke, and its related topic keyword B is placed before topic keyword A.

Once the pair of topic keywords and the pair of punch words are arranged in the optimal order, a bridge is added to the joke. A bridge is a string of words or a template of words that connects the topic sentence or topic keywords to the punch words in a way that results in a natural-sounding, logical-sounding, syntactically well-formed sentence or two, completing the joke. By guiding the audience smoothly and efficiently from the topic sentence or topic keywords to the punch words, the bridge may help set up the audience to be surprised by the punch words, thereby enhancing the humorous effect of the punch words.

In one exemplary embodiment, a bridge is selected at random in step 524 from a list stored in the database. Any one of various random number generators can be used to select a bridge at random from the list in the database.

One exemplary embodiment will draw from a list of bridges like these: "Or maybe," "I have an idea," "That reminds me of," "Instead, how about," "Is that anything like," "Or here's something," "What's next . . .," "Then before you know it," "Makes me think of," "Next you'll be talking about," "Here's a brainstorm," and so forth. This list of bridges is merely an example and is not exhaustive.

Additionally, a filler word or filler words may be added to the selected bridge. The filler word(s) may be selected at random from a list stored in a database. One exemplary embodiment will draw from a list of filler words like these: "Yeah," "Well," "Um," "Oh," "Ha," "Like," "Sure," "You know," "Mmm-hmm," "Hey," and so forth. This list of filler words is merely an example and is not exhaustive. Adding a filler word or filler words may result in a joke that is more natural-sounding.

Once a bridge is selected, the components of the joke of Type #1 are assembled in step 526. First in the joke come the two topic keywords, arranged in their optimal order as detailed above, followed by a question mark. Next comes the bridge that was randomly selected from the list in the database, along with any filler word(s). Last come the two punch words, arranged in their optimal order as detailed above. The result of this assembly process is a rough draft of the joke of Type #1.

In step 528 the rough draft of the joke of Type #1 is edited to make the joke read and sound as much as possible like natural English speech. If the topic keyword in second position is a singular noun, the word "a" or "an" may be inserted before the topic keyword in first position. Then the first letter of the first word of the joke is capitalized. If the punch word in second position is a singular noun, the word "a" or "an" may be inserted before the punch word in first position.

After editing, the completed joke of Type #1 is output in step 530. In implementations the wordplay score of the completed joke is the wordplay score that was calculated in step 520 for the two words related to the topic keywords that became the punch words of the joke. As described below, the wordplay score of the completed joke is used to compare the

funniness of the joke with that of other jokes and to assess the method's confidence in the joke.

Calculation of Wordplay Score

This system for generating jokes utilizes a method for calculating a score for the wordplay between two words or phrases. FIG. 6 shows an exemplary embodiment of a method 600 for calculating a score for the wordplay between two words or phrases. Wordplay between words or phrases, when it occurs in punch word(s), can contribute to the humorous effect of a joke because it is surprising in the context of the joke. Block 610 represents the two words for which the wordplay score is to be calculated; call them word A and word B.

A determination is made as to whether any of a set of excluding rules apply to the pair of words. One example of an excluding rule, at decision step 612, involves what can be called the beginning of each word, by non-limiting example the first 75% of each word (in other implementations it could be the first 5%, 10%, 15%, 20%, 25%, 30%, 35%, 40%, 45%, 50%, 55%, 60%, 65%, 70%, etc., of each word). If word A starts with the beginning of word B or word B starts with the beginning of word A, that pair of words is determined in step 614 to have no wordplay. Other examples of excluding rules, in steps 616 and 618, involve the lemmas and stems of the two words. If the two words share the same lemma or the same stem, that pair of words is determined in step 614 to have no wordplay. Various stemmers and lemmatizers can be used to identify the stems and lemmas of the words. An exemplary embodiment might use the SNOWBALL STEMMER and the WORDNET LEMMATIZER in the NLTK. The reasons for these excluding rules are that in implementations for wordplay to have a humorous effect, it is desirable for the wordplay to elicit surprise in the audience, and the pairing of two words that are fundamentally the same word will generally not elicit surprise.

If the pair of words has not been excluded, a wordplay score is calculated for the pair. In an exemplary embodiment the wordplay score has three components. The first component, in step 620, is the edit distance score, which is the edit distance between the two words. The shorter the edit distance is between the two words, the more surprising it may be to link the two words together in the joke as punch words and the funnier the joke may be.

If the two words are both in a pronunciation dictionary, the edit distance is calculated between the first pronunciation listed for each word in the pronunciation dictionary. If at least one of the words is not in the pronunciation dictionary, the edit distance between the words themselves is calculated instead and a numerical penalty, by non-limiting example 0.5 (which in implementations may be set by the admin and may be any other value, for example 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, etc.), may be factored into the edit distance score. The reason for this numerical penalty is that an edit distance calculation based on how the words are spelled may overstate the degree to which the words are pronounced in a similar way. If the two words rhyme, a numerical bonus of, say, 1.5 (which in implementations may be set by the admin and may be any other value, for example 0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0, etc.) may be factored into their edit distance score. The reason for this numerical bonus is that words that rhyme may in implementations have a greater humorous effect than words that do not rhyme, because words that rhyme end unexpectedly with the same sound(s). Any of various rhyming dictionaries may be used to determine whether the two words rhyme, by non-limiting example James Wenzel's PYTHON rhyming dictionary PHYME. In alternative embodiments, a rhyming dictionary

or a collection of preset rules may be used to determine whether a rhyme is perfect, imperfect, or any other type of rhyme and may be used to assign a different numerical bonus to each type of rhyme. The edit distance score, with any penalty and bonus factored in, becomes one component of the wordplay score. To calculate the edit distance score, an exemplary embodiment might use the CMU Pronouncing Dictionary in the NLTK and any of various implementations of the Levenshtein distance algorithm.

A second component of the wordplay score in an exemplary embodiment, in step 622, is the stop consonant score. The stop consonant score is the total number of stop consonants present in the pair of words as they are spelled, not including stop consonants present in words where the stop consonants are not pronounced as stop consonants. Stop consonants are “b,” “d,” “g,” “k,” “p,” and “t.” The presence of stop consonants in a punch line may in implementations increase the humorous effect of the punch line, so that the more stop consonants in the punch words, the funnier the joke may be. In an alternative embodiment, the stop consonant score may be the total number of stop consonant phonemes present in the pair of words as they are pronounced; a pronunciation dictionary such as the CMU Pronouncing Dictionary would be used in this embodiment.

A third component of the wordplay score in an exemplary embodiment, in step 624, is the alliteration score, which measures the degree to which two words start with the same consonants, the same consonant sounds, or the same phonemes. In an exemplary implementation the alliteration score is 2 if the first phoneme of each of the two words is the same consonant sound, and the alliteration score rises to 3 if, in addition, the second phoneme of each of the two words is the same. These particular points awarded for alliteration are only examples and may be different in other embodiments, and in implementations these may be set by the admin. Alliteration in the punch words, the unexpected repetition of letters or phonemes, may add to the surprise caused by linking the punch words in the joke and may increase the humorous effect of the joke.

The score for the wordplay between the two words may also include additional components. By non-limiting example, the wordplay score may also include a score measuring the degree to which the two words have the same number of syllables, a score measuring the degree to which the two words end with the same phonemes or letters, or a score measuring the degree to which the two words share the same vowel sounds.

In step 626 the wordplay score for the pair of words is calculated and it is output in step 628. In an exemplary embodiment, the wordplay score for the pair of words is calculated as their edit distance score, minus their alliteration score, minus 75% of their stop consonant score. The full stop consonant score is not included in the wordplay score in implementations in order to keep the stop consonant score in balance with the other two components of the wordplay score. In this embodiment, the lower the wordplay score, the better the wordplay.

Example of Generation of Joke of Type #1

Topic sentence: “Archaeologists in Egypt have discovered a pair of socks meant to be worn with sandals from 2,000 years ago.”

Topic keywords selected: “Egypt,” “socks”

Related words with the best wordplay score: “Mideast,” “mittens”

Edit distance score: 4.0

Alliteration score: 3.0

Stop consonant score: 4.0

Wordplay score: $4.0 - 3.0 - (0.75 * 4.0) = -2.0$

Maximum acceptable wordplay score to output a joke: 0.0.

Joke output: “Egypt socks? Then how about Mideast mittens?”

Generation of Joke of Type #2

In addition to a joke of Type #1, the two topic keywords selected as in FIG. 4 can also be used to generate a joke of Type #2 if at least one of the topic keywords is a string consisting of two or more tokens. Examples of such a string are “parole board,” “United States,” and “Santa Claus.” FIG. 7 shows an exemplary embodiment of a method 700 of generating a second type of joke (Type #2).

Block 710 represents the two topic keywords that were outputted in step 446, i.e. topic keyword A and topic keyword B. Decision step 712 determines whether both topic keywords consist of only one token. In one exemplary embodiment, the NLTK has a word tokenizer that will enable the determination. If both topic keywords consist of only one token then the method exits, as shown at step 714, and the method will not attempt to generate a joke of Type #2.

Decision step 716 determines whether topic keyword A consists of more than one token. If it does, in step 718 a list is created of the top fifty, for example (which number may in implementations be set by the admin and may be any number greater than fifty, equal to fifty, or fewer than fifty), words that occur most frequently in contexts where topic keyword B is used. As in step 512, an exemplary embodiment might use the “most_similar” function of GOOGLE’s pretrained WORD2VEC model trained on GOOGLE NEWS data to build the list of most similar words for topic keyword B, which we’ll refer to as the topic keyword B related words. In other implementations, related words, associated words, or most similar words for the topic keyword, including words that occur most frequently in contexts where the topic keyword appears, may also be selected from a database or databases such as, by non-limiting example, a spreadsheet, knowledge base, graph database, relational database, semantic network, entity relationship model, dictionary, thesaurus, lexicon, or WORDNET (created by George A. Miller). Then each of the tokens of topic keyword A is paired in turn with each topic keyword B related word, including topic keyword B itself, and the wordplay score of each pair is calculated as in FIG. 6. The pairing of the topic keyword A token and the topic keyword B related word with the best wordplay score is determined.

If topic keyword A consists of only one token then topic keyword B must consist of more than one token because it has been determined that both keywords do not consist of only one token. So if topic keyword A is determined in decision step 716 to consist of only one token, in step 720 a list is created of the top fifty, for example (which number in implementations may be set by the admin and may be any number greater than, fewer than, or equal to fifty), words that occur most frequently in contexts where topic keyword A is used. Again, as an example, the “most_similar” function of GOOGLE’s pretrained WORD2VEC model trained on GOOGLE NEWS data may be used to build the list of most similar words for topic keyword A, which we’ll refer to as the topic keyword A related words. In other implementations, related words, associated words, or most similar words for the topic keyword, including words that occur most frequently in contexts where the topic keyword appears, may also be selected from a database or databases such as, by non-limiting example, a spreadsheet, knowledge base, graph database, relational database, semantic network,

entity relationship model, dictionary, thesaurus, lexicon, or WORDNET (created by George A. Miller). Then each of the tokens of topic keyword B is paired in turn with each topic keyword A related word, including topic keyword A itself, and the wordplay score of each pair is calculated as in FIG. 6. The pairing of the topic keyword B token and the topic keyword A related word with the best wordplay score is determined.

Decision step 722 determines whether topic keyword B consists of more than one token. If topic keyword B does consist of more than one token then the best wordplay score is determined, in step 720, from among the pairings of all tokens of topic keyword B and topic keyword A related words, including topic keyword A itself. If topic keyword B consists of only one token then no pairing of the token of topic keyword B and the topic keyword A related words is evaluated in decision step 724.

Decision step 724 determines whether any pairing of topic keyword token and topic keyword related word has been found in step 718 and step 720 that has a wordplay score which is better than a certain preset threshold, which may be determined by the admin in implementations and may be set to any desired value. If such a pairing has not been found, then the method exits as shown at step 726 and the method will not attempt to generate a joke of Type #2.

If at least one pairing of topic keyword token and topic keyword related word has been found with a wordplay score that is better than a certain preset threshold, then the pairing with the best wordplay score is determined in step 728. The topic keyword containing the token that is in the pairing that yielded the best wordplay score becomes the host string for the punch word to be created for a joke of Type #2.

The punch word for the joke of Type #2 is created in step 730. To create the punch word, the topic keyword related word in the pairing determined to have the best wordplay score is substituted in the host string for the token in the host string with which it was determined to have the wordplay.

In step 732 the word that was inserted into the host string in order to create the punch word has its first letter capitalized if necessary in order to match the capitalization of the token it replaced. Matching the capitalization ensures that the completed joke will read as much as possible like standard English.

The components of the joke of Type #2 are assembled in step 734. First in the joke comes the original host string, i.e. the host string before the word was inserted into it to create the punch word. The first letter of the original host string is capitalized because it starts a sentence. A question mark is inserted after the original host string, and then the words "More like." Next in the joke comes the punch word that was created in step 730. Last come a comma and then the words "am I right?" The result of this assembly process is the completed joke. These are representative examples, and other phrases conveying meaning similar to "More like" and "am I right" could be used in other implementations to create a joke of Type #2. In an alternative implementation, a bridge selected at random from a list stored in a database, as described for assembling a joke of Type #1, may be added between the original host string and the punch word to assemble the completed joke of Type #2. A filler word or filler words may also be added to the selected bridge as described earlier.

The completed joke of Type #2 is output in step 736. The wordplay score of the completed joke is the wordplay score that was calculated in step 728 for the word and token that were used to create the punch word. As described below, the wordplay score of the completed joke is used to compare the

funniness of the joke with that of other jokes and to assess the method's confidence in the joke.

Example of Generation of Joke of Type #2

Topic sentence: "A new study says eating egg yolks after the age of 40 is as bad for you as smoking."

Topic keywords selected: "egg yolks," "smoking"

Topic keyword tokens: "egg," "yolks"

The topic keyword token and topic keyword related word with the best wordplay score: "yolks," "smokes"

Edit distance score: 0.5 (This score includes a bonus of 1.5 because the words rhyme.)

Alliteration score: 0.0

Stop consonant score: 2.0

Wordplay score: $0.5 - 0.0 - (0.75 * 2.0) = -1.0$

Maximum acceptable wordplay score to output a joke: 0.0.

Joke output: "Egg yolks? More like egg smokes, am I right?"

Generation of Joke of Type #3

The two topic keywords selected as in FIG. 4 can also be used to generate a joke of a third type (Type #3), in which the punch word is a portmanteau word. First a list of related words is compiled for topic keyword A and a separate list of related words is created for topic keyword B. As before, an exemplary embodiment might use the "most_similar" function of GOOGLE's pretrained WORD2VEC model trained on GOOGLE NEWS data to build the lists of related words, which may include noun-phrase chunks and verb-phrase chunks. In other implementations, related words, associated words, or most similar words for each topic keyword, including words that occur most frequently in contexts where the topic keyword appears, may also be selected from a database or databases such as, by non-limiting example, a spreadsheet, knowledge base, graph database, relational database, semantic network, entity relationship model, dictionary, thesaurus, lexicon, or WORDNET (created by George A. Miller).

Then the punch word is created by substituting a word related to one topic keyword for one or more syllables of a word related to the other topic keyword with which it has adequate wordplay. The adequacy of the wordplay between the word being inserted and the syllable or syllables it is replacing is determined by calculating their wordplay score as shown in FIG. 6. After the punch word has been assembled, a bridge is created, including any filler word(s), and is used to connect the topic sentence to the punch word to complete the joke. In implementations the wordplay score of the completed joke is the wordplay score associated with the word and syllables that were substituted for one another to create the punch word.

FIG. 8 shows an exemplary embodiment of a method 800 of creating a bridge and outputting a joke of the third type (Type #3). Block 810 represents the punch word of the joke of Type #3. Step 812 identifies the punch word root, which in implementations is the topic keyword related word that comprises at least half of the punch word (though in other implementations it may comprise greater or less than half of the punch word and may be set by the admin). Step 814 extracts from a database or corpus of documents (internal to the system or a third party provided database/corpus accessed by the system through the Internet) the punch word root definition, which is a short phrase which defines the punch word root. To extract the punch word root definition, an exemplary embodiment might use the method described in U.S. Pat. No. 7,376,551 B2, "Definition Extraction," issued May 20, 2008, listing as first inventor Kevin R. Powell, the entire disclosure of which is incorporated herein

21

by reference. In implementations step 816 selects a bridge template at random from a list stored in a database. Each bridge template assigns the punch word root definition to the punch word.

Here are some non-limiting examples of bridge templates: 5

"So that means a new [punch word root definition] is [punch word]."

"So how about a new [punch word root definition]: [punch word]."

"Sounds like the new [punch word root definition] is [punch word]."

"They call the new [punch word root definition] [punch word]."

"And maybe a new [punch word root definition] could be [punch word]."

"That's why a new [punch word root definition] is [punch word]."

This list of bridge templates is merely an example and is not exhaustive.

The punch word root definition and punch word are added to the selected bridge template in step 818, assembling and completing the joke. In other implementations, a bridge that does not incorporate a punch word root definition may be selected at random from a list stored in a database and used in connection with the punch word to assemble the completed joke. A filler word or filler words may also be added to the selected bridge as described earlier. The completed joke is output in step 820.

Example 1 of Generation of Joke of Type #3

Topic sentence: "For the Olympics, the mayor of Rio wants Pokemon Go to come to Brazil."

Topic keywords selected: "Olympics," "Pokemon Go"

Words related to the topic keywords: "Zika" [the mosquito-borne virus], "Pikachu"

Punch word: "Zikachu"

Punch word root and definition: "Pikachu," "fictional creature" 40

Completion of joke with bridge: "So how about a new fictional creature: Zikachu."

Example 2 of Generation of Joke of Type #3

Topic sentence: "Dow Chemical is turning raw sewage into drinking water."

Topic keywords selected: "sewage," "drinking water"

Words related to the topic keywords: "poo," "Poland Spring" 50

Punch word: "Pooland Spring"

Punch word root and definition: "Poland Spring," "brand of bottled water"

Completion of joke with bridge: "They call the new brand of bottled water Pooland Spring." 55

Example 3 of Generation of Joke of Type #3

Topic sentence: "A new website for Disney fans who want to date is called Mouse Mingle." 60

Topic keywords selected: "Disney," "date"

Words related to the topic keywords: "Cinderella," "Tinder"

Punch word: "Tinderella"

Punch word root and definition: "Tinder," "online dating app" 65

22

Completion of joke with bridge: "And maybe a new online dating app could be Tinderella."

Example 4 of Generation of Joke of Type #3

Topic sentence: "America's national forests are being severely damaged by the illegal growing of marijuana."

Topic keywords selected: "national forests," "marijuana"

Words related to the topic keywords: "Smokey the Bear," "toke" 10

Punch word: "Tokey the Bear"

Punch word root and definition: "Smokey the Bear," "advertising icon"

Completion of joke with bridge: "That's why a new advertising icon is Tokey the Bear." 15

Generation of Joke of Type #4

The two topic keywords selected as in FIG. 4 can also be used to generate a joke of Type #4, in which the punch word is a word chunk such as a noun phrase chunk. First a list of related words and word chunks is compiled for topic keyword A and a separate list of related words and word chunks is created for topic keyword B. As before, an exemplary embodiment might use the "most_similar" function of GOOGLE's pretrained WORD2VEC model trained on GOOGLE NEWS data to build the lists of related words and word chunks. Related, associated, or most similar words and word chunks for each topic keyword, including words and word chunks that occur most frequently in contexts where the topic keyword appears, may also be selected from a database or databases such as, by non-limiting example, a spreadsheet, knowledge base, graph database, relational database, semantic network, entity relationship model, dictionary, thesaurus, lexicon, or WORDNET (created by George A. Miller). 30

Then the punch words are created by substituting a word related to one topic keyword for one or more words contained in a word chunk related to the other topic keyword. The word being inserted and the word or words it replaces are the same part of speech—nouns or adjectives for example—and also have adequate wordplay. The adequacy of the wordplay between the word being inserted into the word chunk and the word or words it is replacing is determined by calculating their wordplay score as shown in FIG. 6. 40

After the punch words have been assembled, a bridge is created and is used to connect the topic sentence to the punch words to complete the joke. FIG. 8 shows an exemplary embodiment of the process of creating a bridge and outputting a joke. The punch word root definition and punch word are added to the selected bridge template in step 818, assembling and completing the joke. In other implementations, a bridge that does not incorporate a punch word root definition may be selected at random from a list stored in a database and used in connection with the punch word to assemble the completed joke. A filler word or filler words may also be added to the selected bridge as described earlier. The completed joke is output in step 820. In implementations the wordplay score of the completed joke is the wordplay score associated with the words that were substituted for one another to create the punch words. 45

Example 1 of Generation of Joke of Type #4

Topic sentence: "Yesterday seven more cases of measles were linked to Disneyland."

Topic keywords selected: "measles," "Disneyland"

23

Words and word chunks related to the topic keywords: “rash,” “Splash Mountain”
 Punch words: “Rash Mountain”
 Punch word root and definition: “Splash Mountain,” “log-flume ride”
 Completion of joke with bridge: “So that means a new log-flume ride is Rash Mountain.”

Example 2 of Generation of Joke of Type #4

Topic sentence: “Two Wisconsin men were arrested in the theft of 100,000 diapers.”
 Topic keywords selected: “arrested,” “diapers”
 Words and word chunks related to the topic keywords: “petty larceny,” “potty”
 Punch words: “potty larceny”
 Punch word root and definition: “petty larceny,” “crime”
 Completion of joke with bridge: “They call the new crime potty larceny.”

Example 3 of Generation of Joke of Type #4

Topic sentence: “A teacher was fired for leading third graders in the Nazi salute.”
 Topic keywords selected: “third graders,” “Nazi salute”
 Words and word chunks related to the topic keywords: “Duck, Duck, Goose,” “goose-step”
 Punch words: “Duck, Duck, Goose-step”
 Punch word root and definition: “Duck, Duck, Goose,” “children’s game”
 Completion of joke with bridge: “Sounds like the new children’s game is Duck, Duck, Goose-step.”
 Confidence Ranking and Joke Output

In implementations methods for generating jokes may include generating multiple jokes using the same keywords and then determining which joke has the highest ranking and outputting that joke. For example, FIG. 9 shows an exemplary embodiment of a process (method) 900 of ranking and outputting one of multiple jokes generated by the above methods. Block 910 represents the jokes of Type #1, Type #2, Type #3, and/or Type #4 that were generated. In alternative embodiments, the bridge words for those jokes may have been generated by a machine learning language model as shown in FIGS. 10, 11, 12, and 13.

Decision step 912 determines whether the method has created any jokes. If the method has created no jokes then the method exits, as shown at step 914, and the method will not output a joke. Decision step 916 determines whether the method has created only one joke. If the method has created only one joke, step 918 tests whether the wordplay score of the joke meets a preset threshold (which in implementations may be set by the admin). If it does, that means the method has sufficient confidence in the joke and the joke is output in step 920. If the wordplay score of the joke does not meet the preset threshold then the method exits, as shown at step 922, and no joke is output.

In step 924 any jokes of Type #1, Type #2, Type #3, and/or Type #4 are ranked according to their wordplay score and the joke that is ranked highest is selected. Step 926 tests whether the wordplay score of the selected joke meets a preset threshold (which in implementations may be set by the admin). If the wordplay score of the selected joke does not meet the preset threshold then the method exits, as shown at step 928, and no joke is output. If the wordplay score of the selected joke does meet the preset threshold, that

24

means the method has sufficient confidence in the selected joke and the selected joke is output in step 930.

Alternative Embodiment of Method Using Machine Learning

In generating the jokes of Type #1, Type #2, Type #3, and Type #4 as described above, bridges were added that incorporated pre-written word templates or pre-written word strings selected from lists stored in the one or more databases of the system (or stored in remote databases accessed by one or more servers of the system through the Internet). In alternative embodiments, the bridge for jokes of those types will not be pre-written or “canned” but will be generated by a machine learning language model. A machine learning language model is a natural language processing model which learns to predict the next word in a sentence. Bridges generated by a machine learning language model may allow the jokes generated by the method to be more varied and therefore more natural-sounding in the context of a lengthy interaction with the user.

FIG. 10 illustrates an example high-level flowchart of an alternative embodiment of a method 1000 for generating jokes, which jokes may be original, contextually integrated jokes based on text that has been obtained from an external source (such as the external sources described above, including any user devices and/or any news or other website from which the system may receive/gather information). At step 1010, the method receives text from a human interaction device and/or from a news source/website etc.; this process is the same process as that described for steps 310 through 314 inclusive. At step 1012, the method selects from that text a topic sentence from which the joke generation process will begin; this process is the same process as that described for steps 316 through 320 inclusive. From that topic sentence, a pair of topic keywords is selected in step 1014; this process is the same process as that described for steps 410 through 446 inclusive. From those topic keywords, a punch word or punch words which may trigger a laugh from the user are created in step 1016. These punch words may be created as described for steps 510 through 520 inclusive, as described for steps 710 through 730 inclusive, as described for the generation of a joke of Type #3 or of Type #4, or by any other process that creates a surprising or unexpected word or phrase based on the topic keyword(s) selected from a topic sentence. Decision step 1018 determines whether the wordplay score of any punch word(s) meets a preset threshold (which in implementations may be set by the admin). If the wordplay score of none of the punch word(s) meets the preset threshold then the method exits, as shown in step 1020, and no joke is output. If the wordplay score of any of the punch word(s) meets the preset threshold then step 1022 selects the punch word(s) with the highest-ranked wordplay score.

In step 1024 a machine learning language model is built. The forward version of the machine learning language model is used with a forward beam search strategy in step 1026 to generate a list of candidates for bridges that may naturally connect the topic sentence to the punch word(s). In step 1028, the topic sentence from step 1012, the list of bridge candidates from step 1026, and the punch word(s) selected in step 1022 are concatenated in that order to form a list of candidates for jokes. In step 1030 the backward version of the machine learning language model is used to select the top, most likely, joke candidate from the list of joke candidates compiled in step 1028. The top joke candidate is output in step 1032. In an exemplary embodiment, the

text source output in step 1032 may then be used as the input to a text-to-speech converter such as, for example, GOOGLE CLOUD TEXT-TO-SPEECH, though in other implementations it could simply be visually displayed on a user interface of an end user's computing device/phone etc. The selected joke may accordingly be delivered in audio format or visual format to the user.

An alternative embodiment of the process outlined in FIG. 10 may use the forward version of the machine learning language model with a forward beam search strategy and the backward version of the machine learning language model with a backward beam search strategy and may alternately perform forward beam searches starting with the topic sentence and backward beam searches starting with the punch word(s), searching through all possible bridge candidates up to a maximum specified bridge length and determining which bridge candidate maximizes the probability of the entire joke.

Another alternative embodiment of the process outlined in FIG. 10 may use a bidirectional machine learning language model to perform the forward or backward searches. Still another alternative embodiment of the process outlined in FIG. 10 may use a pretrained machine learning language model to generate the bridge candidate(s) forward from the topic sentence, backward from the punch word(s), or in both directions. Before being used to generate the bridge candidate(s), the pretrained machine learning language model may be fine-tuned on an additional dataset or datasets, by non-limiting example, a dataset of jokes.

To illustrate in more detail the process broadly outlined in FIG. 10, detailed examples of block diagrams that describe steps 1024 through 1032 of this alternative embodiment of the method follow. FIG. 11 shows an exemplary embodiment of step 1024, which is the process (method) 1100 of building the machine learning language model for the method of FIG. 10. Again, some or all of the following steps may be performed at the user device level using installed software and other aspects may be performed using the DB server and/or other servers of the system. In step 1110, a general (and in implementations a very large) language dataset is read into the system. In an exemplary embodiment, this dataset may be Stephen Merity's WIKITEXT-103 language modeling dataset, which is a collection of over 100 million words extracted from the set of verified Good and Featured articles on the English WIKIPEDIA websites. A general and very large dataset such as this may result in an English language model with the world knowledge and the deep understanding of grammar, semantics, and other elements of natural language that are useful for the method to generate effective bridges for jokes.

Once the very large language dataset is read into the system it is prepared for use in training a universal language model. In step 1112, in an exemplary embodiment, the dataset text is tokenized and a dictionary compiled which comprises the vocabulary of the dataset text and the integer index of each word in the vocabulary. In an exemplary embodiment, this text preprocessing and other tasks of building and training the language models and using them to generate jokes may be accomplished using the neural network library KERAS with the machine learning library TENSORFLOW as the backend engine. In step 1114, in an exemplary embodiment, the lines of dataset text are turned into sequences of index integers which are then used to create a list of separate sequences of varying length to be used in training the models. In step 1116, in an exemplary

sequence in the list. And in step 1118, in an exemplary embodiment, the pre-padded sequences are converted to the form necessary for use in training the language model. In an exemplary embodiment, the NUMPY library for PYTHON is used to create an array from the list of pre-padded sequences and each sequence is split into input and output elements, where the output element is the last element in the sequence and is converted to a binary class matrix marking the corresponding word in the vocabulary.

Now that the training data has been prepared, the universal language model is defined in step 1120. In an exemplary embodiment, the universal language model may be a unidirectional recurrent neural network (RNN) such as a long short-term memory network (LSTM). In an exemplary embodiment, the LSTM model may have an embedding layer, multiple LSTM layers, and an output layer that uses the softmax activation function, and may also have an argument which determines whether the model reverses the input sequences. In step 1122, the universal language model is trained using the prepared training data that has not been reversed. This yields a forward universal language model that can be used to predict the next word in a sentence. In step 1124, the universal language model is trained using the prepared training data that has been reversed. This yields a backward universal language model that can be used to predict the previous word in a sentence.

In an alternative embodiment, instead of building universal language models as described for steps 1110 through 1124 inclusive, where the word embeddings are learned by the models themselves during training, the method may use for universal language models a model or models with pretrained word embeddings that have been learned elsewhere. In an exemplary embodiment, the forward universal language model may have an embedding layer that has been loaded with an embedding matrix computed from the GLOVE (Global Vectors for Word Representation) model that has been trained on a year 2014 dump of English WIKIPEDIA websites and GIGAWORD 5; further learning by the forward universal language model would then take place on top of that pretrained embedding. In other alternative embodiments, a version of GOOGLE's BERT (Bidirectional Encoder Representations from Transformers) that has been pretrained on English WIKIPEDIA websites and BOOKSCORPUS, or a version of another pretrained language model, may be used as a forward universal language model or a backward universal language model.

In step 1126 a personalized user language dataset may be obtained, read into the system, and prepared as in steps 1112 through 1118 inclusive for use in performing transfer learning with the forward universal language model and/or with the backward universal language model. In an exemplary embodiment, the personalized user language dataset may be obtained by scraping the user's most frequently visited websites, websites in the user's online search and browsing history, social media posts made by the user, social media posts "liked" by the user, the user's email or text messages, the user's communications with an AI virtual assistant, and/or other personal data sources. In step 1128, the personalized user language dataset may be used to refine the forward universal language model, which may have been pretrained in step 1122 on the more general language dataset. In this way a personalized language model may be developed more quickly that can more skillfully predict words that reflect the user's knowledge base, beliefs, and attitudes. In step 1130, the personalized user language dataset may also be used in a similar way to refine the

backward universal language model, which may have been pretrained in step 1124 on the more general language dataset.

In step 1132 a joke dataset may be obtained, read into the system, and prepared as in steps 1112 through 1118 inclusive for use in performing additional transfer learning with the forward personalized language model and/or with the backward personalized language model. In an exemplary embodiment, the joke dataset is obtained by scraping websites that aggregate jokes that have been performed in the opening monologues of late-night comedy/talk television shows. Those monologue jokes may be useful for training language models to generate jokes because the monologue jokes tend to be relatively short and tend to have structures that have been devised by professional comedy writers so as to maximize the humor potential of the jokes. In step 1134, the joke dataset may be used to fine-tune the forward personalized language model, which was built in step 1128, for the objective of building a forward joke model. This forward joke model, which may help to generate effective bridges for jokes, is output as the forward bridge model in step 1138. In step 1136, the joke dataset may also be used to fine-tune the backward personalized language model, which was built in step 1130, for the objective of building a backward joke model. This backward joke model, which may help to generate effective bridges for jokes, is output as the backward bridge model in step 1140. In alternative embodiments, the joke dataset may be used to train or fine-tune the forward universal language model, the backward universal language model, or any other language model used in the joke generation process.

Now that the forward bridge model and the backward bridge model have been built, they are used in the joke generation process as outlined in a high-level way in FIG. 10, steps 1026 through 1032 inclusive. To illustrate step 1026 in more detail, FIG. 12 shows an exemplary embodiment of the process (method) 1200 of using the forward bridge model to generate a list of some of the bridges most likely to follow a given topic sentence. A forward beam search strategy is used to return a list of the bridges that are predicted to be the most likely to follow a given topic sentence given a specified beam width and a specified maximum length of the bridge in words. In implementations, the beam width and the maximum bridge length in words may be preset or may be input by the admin in step 1210 using the admin computer. In step 1212, the topic sentence that was output at step 320 by the process shown in FIG. 3 is obtained. In step 1214, the topic sentence and one of the top-[beam width] bridge candidates are concatenated in the order of topic sentence first, then bridge candidate; if no bridge candidates have been identified yet at this point in the process, the topic sentence is not concatenated with anything. In step 1216, the forward bridge model is used to predict the probability that each word in the dictionary of the model will follow the concatenation of the topic sentence and the current top-[beam width] bridge candidate, if any. Decision step 1218 shows that steps 1214 and 1216 are repeated until the topic sentence and each of the top-[beam width] bridge candidates have been concatenated and the forward bridge model has been used to calculate the probability that each word in the model dictionary would follow each concatenation of topic sentence and top-[beam width] bridge candidate.

In step 1220, a list is made of the top-[beam width] most likely next words in the bridge as predicted by the forward bridge model, together with the bridge candidates that, when concatenated with the topic sentence, were used to predict

those top-[beam width] most likely next words. In step 1222, each of the top-[beam width] most likely next words in the bridge is concatenated to the end of whichever bridge candidate was used to predict it, yielding a list of [beam width] new bridge candidates, which are added to a list of all top bridge candidates. Decision step 1224 shows that steps 1214 through 1222 inclusive are repeated, each time concatenating the topic sentence with top-[beam width] bridge candidates that are one word longer than the previous top-[beam width] bridge candidates, and each time adding [beam width] new bridge candidates to the list of all top bridge candidates; steps 1214 through 1222 inclusive are repeated until the length of each of the [beam width] newest bridge candidates that were added to the list of all top bridge candidates is the maximum bridge length. The list of all top bridge candidates generated by this process is output in step 1226; the number of bridge candidates on that list is equal to the product of the beam width and the maximum bridge length. The steps shown in FIG. 12 (as with steps shown in the other drawings) may be performed by one or more of the aforementioned servers of the system 1400.

Now to illustrate steps 1028 through 1032 in more detail, FIG. 13 shows an exemplary embodiment of the process (method) 1300 of using the backward bridge model to choose from the list of all top bridge candidates the best bridge to smoothly connect to the given topic sentence any punch word or punch words that were generated from the topic sentence using the methods as described earlier. Although in implementations the topic sentence, the top bridge candidates, and the punch word(s) may still all be in the form of sequences of index integers, "word" will be used at times instead of "index integer" throughout this description as a convenience.

The topic sentence that was output at step 320 by the process shown in FIG. 3 is obtained in step 1310. The list of all top bridge candidates that was output at step 1226 by the process shown in FIG. 12 is obtained in step 1312. And the punch word(s), if any, that were generated and selected by the process outlined in steps 1016 through 1022 inclusive are obtained in step 1314. In an alternative embodiment, when the punch word(s) have been created in the process of generating a joke of either Type #3 or Type #4, the associated punch word root may be used as the punch word(s) in steps 1314 through 1332 inclusive.

In step 1316, the topic sentence, the punch word(s), and each top bridge candidate in turn are concatenated in the order of topic sentence, bridge candidate, punch word(s), thereby forming a list of joke candidates. Each joke candidate in that list is reversed in step 1318, forming a list of reversed joke candidates.

In step 1320, one of the reversed joke candidates is taken from the list of reversed joke candidates and prepared for input into the backward bridge model that was output in step 1140. In an exemplary embodiment, the reversed joke candidate is prepared by pairing each word in the reversed joke candidate with the list of words preceding it in the reversed joke candidate, that list of words having been pre-padded with zeroes, and then converting the two elements of each pair into arrays using the NUMPY library for PYTHON.

In step 1322, the backward bridge model output in step 1140 is used to predict the conditional probability that each word in the vocabulary of the model will be the next word at each position in the reversed joke candidate, given the words that precede that position in that reversed joke candidate.

In step 1324, the conditional probability of occurrence of a word that is actually present in the reversed joke candidate,

given the words that precede it in the reversed joke candidate, is obtained from the conditional probability predictions made by the backward bridge model in step 1322. In step 1326, the conditional probability obtained for the word in step 1324 is multiplied by the product of the conditional probabilities of occurrence of the words that precede it, using the chain rule for probability. The chain rule for probability may be represented as follows, where P is the probability of joint occurrence and $A_1, \dots, A_n$ is an indexed collection of random variables, words in this case:

$$P(A_n \dots A_1) = P(A_n | A_{n-1}, \dots, A_1) * P(A_{n-1}, \dots, A_1)$$

In an exemplary embodiment, the logs of the probabilities are added instead of the probabilities being multiplied because the probabilities may be very small. Decision step 1328 shows that steps 1324 and 1326 are repeated for each word in the reversed joke candidate until the probability of occurrence of the entire reversed joke candidate is computed using the chain rule for probability.

Decision step 1330 shows that steps 1320 through 1328 inclusive are repeated until the probability of occurrence of each and every reversed joke candidate has been calculated using the backward bridge model and the chain rule for probability. In step 1332, the reversed joke candidate that has the highest probability of occurrence is selected. This is the reversed joke candidate that contains the top bridge candidate that has been determined to most naturally link the given topic sentence to the given punch word(s). In step 1334 this highest-probability reversed joke candidate is reversed back to its forward state and a filler word or filler words may be added as described earlier. If a punch word root has been used as the punch word(s) in steps 1314 through 1332 inclusive, the punch word(s) are then substituted for the punch word root in the selected joke in step 1334. The selected joke is output in step 1336.

In an exemplary embodiment, text source 1336 may then be used as the input to a text-to-speech converter such as, for example, GOOGLE CLOUD TEXT-TO-SPEECH, or it may alternatively be displayed as text on a user interface of an end user's computing device/phone, etc. The selected joke may accordingly be delivered in audio format or visual format to the user. In implementations, the text-to-speech converter may be trained or fine-tuned so as to generate speech in a manner that may increase the funniness of the spoken joke. By non-limiting example, the text-to-speech converter may be trained or fine-tuned to generate output that incorporates certain acoustic, auditory, or prosodic features of spoken jokes such as the loudness of certain parts of the joke, the intonation of certain parts of the joke, the pauses at certain points in the joke, the speech rate of certain parts of the joke, the pitch of certain parts of the joke, and so forth. In implementations these acoustic, auditory, or prosodic features of spoken jokes may be learned or extracted from audio samples of people uttering jokes, or set by the admin, or obtained by some other means.

An alternative embodiment of the process outlined in FIG. 13 may use a language model such as, by non-limiting example, a version of GOOGLE's BERT to fill with a bridge the gap between the given topic sentence and any punch word(s) that were generated from the topic sentence using the methods as described earlier. A filler word or filler words may be added as described earlier to the bridge that was generated by the language model.

The methods disclosed herein are based on using wordplay to link two topic keywords and words related to them and thereby produce a humorous effect. Alternative implementations of these methods may link two topic keywords

and words related to them in ways that do not depend on wordplay. As one example, the links between two topic keywords and words related to them could be created using reasoning informed by commonsense knowledge and by the anticipated knowledge, beliefs, and attitudes of the intended audience of the joke to be generated. Exemplary embodiments might use the Allen Institute for AI's COMET knowledge base construction engine or ATOMIC atlas of commonsense reasoning or another machine reasoning system to perform the reasoning tasks required to create links between two topic keywords and words related to them. Other embodiments might create a link between two topic keywords and words related to them by using a graph database, relational database, or common sense knowledge base such as, by non-limiting example, CYC (created by Douglas Lenat). Still other implementations might use a language model like, by non-limiting examples, a version of GOOGLE's BERT or a sequence-to-sequence model to convert a topic sentence or sentences into a set of punch words, a set of bridge words, or a set of both bridge words and punch words, where the punch words or the bridge words may or may not depend on wordplay.

The methods disclosed herein may be used for joke generation in a variety of settings. For example, the system in implementations may simply begin a joke with an introductory sentence (based on some news headline or the like) and then deliver the generated joke. This implementation could be used, for example, by a robot or some computing device delivering a standup comedy routine, where the routine is not necessarily affected by any audience input. In other implementations, however, the system may generate jokes in response to user input, for example a user speaking some sentence or group of sentences or asking some question either in audio or text format to APPLE's SIRI, GOOGLE ASSISTANT, AMAZON's ALEXA, a chatbot, app, website, or the like, and the virtual assistant, chatbot, or the like delivering a joke in response (or a joke in addition to the requested information) in audio and/or text format. (This implementation could also be used for an interactive standup comedy routine.) An example of this is shown in FIG. 15, in which the user asks a virtual assistant (such as APPLE's SIRI, AMAZON's ALEXA, or GOOGLE ASSISTANT) for directions to a marathon and the virtual assistant delivers a joke related to the request, but also provides a response to the request for information. In FIG. 15 this is shown as a typed interaction—the user typing a question and the virtual assistant responding similarly with a text response—but as indicated above in implementations such an interaction could be partially or fully in audio format, such as with the end user asking a question in audio format to the virtual assistant and/or the virtual assistant delivering the joke and/or the requested information in audio format (or some combination of audio and visual).

In this way the systems and methods could be used as a module to give a virtual assistant or chatbot a more engaging personality, which may make the user more comfortable when interacting with it. The systems and methods could also be used to give a social robot a sense of humor, which would make the robot more humanlike and enable the robot to be a more congenial companion, which would help reduce the user's loneliness and, in that way, may have significant health benefits. The systems and methods do not need to be used in standup comedy, virtual assistant, chatbot, or social robot settings, however, and may generate jokes for any type of setting. Still another implementation could be a humor-challenged user having a concealed earpiece and microphone which allow the user to deliver ad-libbed jokes based

on his/her contextual setting and/or real-time dialogue, the generated jokes, or perhaps only the generated punch word(s), being presented to him/her discreetly via the earpiece. Such systems and methods could be used in all sorts of social gatherings, on dates, in public speaking, and so forth, and may allow the user to not only have an immediate boost of confidence but, over time, may assist the user in the long-term development of a better sense of humor.

Any implementation of the joke generating systems may be integrated with an affective sensing system, an emotion detection system, or a sentiment analyzer, as a non-limiting example the SENTIMENT ANALYSIS feature of GOOGLE CLOUD NATURAL LANGUAGE API, which would evaluate the user's emotional state and detect whether joke-making would be acceptable to the user at that particular time. If the affective sensing system, emotion detection system, or sentiment analyzer predicts that the user may not react favorably to a joke at that time, the joke generating systems would refrain from outputting to the user whatever joke it may have created. The joke generating systems may also refrain from outputting a joke to the user if it predicts, based on previous interactions with the user, that the user would not appreciate a joke at that time.

Other implementations of joke generating systems may be used in reverse to recognize jokes. In an exemplary implementation of this sort, a chatbot could output a sentence to a user, to which the user would respond by inputting some text or speech to the chatbot. The chatbot would then compute the wordplay score of the final two or more words of the user's input using the methods described above and, if the wordplay score meets a certain preset threshold, the chatbot would respond with the text "ha-ha-ha," the sound of laughter, or some other output indicating that it recognized a joke from the user. In a similar manner, the chatbot could extract the final two or so words of the user's input text or speech, detect whether any of those words is a multi-token string that is not in its vocabulary, compute whether any of the tokens in that out-of-vocabulary multi-token string is closely related to a topic keyword in the output of the chatbot to which the user had responded, compute the wordplay score between the closely related token and the token which it had replaced in the original, in-vocabulary version of the multi-token string and, if the wordplay score meets a certain preset threshold, the chatbot would respond with some output indicating that it had recognized a joke from the user. Also similarly, the chatbot could extract the final few words of the user's input, employ the reverse of the techniques described above for generating a joke of Type #3 to detect whether any of the final few words of the user's input is a portmanteau word, and if the wordplay score associated with the word and syllables that were substituted for one another to create the portmanteau word meets a certain preset threshold, the chatbot would respond with some output indicating that it had recognized a joke from the user.

For example, reversing the example of FIG. 15, the user interface of FIG. 15 could provide an initial sentence to the user via a text or audio prompt from the virtual assistant of the mobile device, such as "I'll give you directions to the New York City Marathon," to which the user could reply (by audio or text input) "The one race in America that has yet to be insulted by Mel Gibson," and the virtual assistant could reply "Ha ha!" or "Good one" or the like.

It is also pointed out here that such systems and methods for recognizing jokes from a user do not need to begin with a prompt from the system. For example, the user could, without any prompt, simply provide a joke or punchline to

the chatbot or virtual assistant, and the above joke recognition techniques may be used to determine that the user has provided a joke or punchline and respond with laughter or some other indication of recognition of the user's joke.

As indicated to some extent above, a system implementing these joke generating methods may include one or more or all of the following: any number of components communicatively coupled together, including one or more admin computing devices, one or more admin servers, one or more application servers, one or more database servers, one or more databases, one or more third party databases, one or more third party database servers, one or more third party application servers, one or more third party datasets or libraries, one or more web servers, a telecommunications network, one or more user devices including mobile phones, tablets, desktop computers, laptops, smart watches, smart speakers, smart glasses, robots, other human interface devices, and the like; each device/component may have its own computing elements including storage, computer processors, software, and additional components as described; and the different steps of the methods described herein may be divvied up between the different components in a variety of ways between the different servers, computing devices, and so forth, as determined by the practitioner of ordinary skill in the art.

In implementations methods for generating jokes as described herein include generating jokes that include the following elements organized in the following order: topic word(s)/keyword(s), bridge word(s), and punch word(s); or topic(s), bridge word(s), punch word(s).

In implementations methods and systems disclosed herein may be used to help provide a conversational chatbot with a sense of humor. In implementations the systems and methods may be used in connection with existing conversational capabilities of a virtual assistant (like APPLE's SIRI, AMAZON's ALEXA, or GOOGLE ASSISTANT). In other words, the methods/systems herein may be used as an add-on to the existing virtual assistant and use whatever interface the virtual assistant uses. In such an embodiment the methods/systems may enable the virtual assistant to ad-lib occasional jokes and output them to the user along with all the other output that the virtual assistant would normally be supplying. In other implementations, the systems and methods may be used as add-ons to an existing open-domain chatbot, entertainment chatbot, chit-chat chatbot, or conversational agent like, by non-limiting example, GOOGLE's MEENA. In such an embodiment, the methods/systems herein may enable the open-domain chatbot to ad-lib occasional jokes and output them to the user along with the other output that the chatbot would normally be supplying.

In places where the phrase "one of A and B" is used herein, including in the claims, wherein A and B are elements, the phrase shall have the meaning "A or B." This shall be extrapolated to as many elements as are recited in this manner; for example the phrase "one of A, B, and C" shall mean "A, B, or C," and so forth.

In places where the description above refers to specific embodiments of systems and methods for generating jokes, one or more or many modifications may be made without departing from the spirit and scope thereof. Details of any specific embodiment/implementation/interface described herein may, wherever possible, be applied to any other specific implementation/embodiment/interface described herein.

33

What is claimed is:

1. A system for generating jokes, comprising:
 one or more databases;
 one or more servers communicatively coupled with the
 one or more databases, and;
 a first computing device communicatively coupled with at
 least one of the one or more servers through a tele-
 communications network;
 wherein the one or more servers are configured to receive
 text from an external source communicatively coupled
 with the one or more servers through the telecommu-
 nications network and, in response, select one or more
 topic keywords, generate one or more punch words
 using words stored in the one or more databases which
 are related to the one or more topic keywords, and add
 one or more bridges to the one or more punch words to
 generate one or more jokes by generating one or more
 punchlines, and communicate a signal to the first com-
 puting device;
 wherein the first computing device is configured to, in
 response to receiving the signal, display or speak one of
 the one or more jokes; and
 wherein the one or more servers are configured to gen-
 erate the one or more punch words based at least in part
 on one of:
 one or more wordplay scores; and
 one or more of: data representing common knowledge,
 data representing anticipated knowledge of an
 intended audience, data representing beliefs of the
 intended audience, and data representing attitudes of
 the intended audience.
2. The system of claim 1, wherein the external source is
 the first computing device.
3. The system of claim 2, wherein the received text is text
 typed or spoken to a virtual assistant using the first com-
 puting device.
4. The system of claim 1, wherein the external source is
 a third party server and is not the first computing device.
5. The system of claim 1, wherein the one or more servers
 are further configured to select a topic sentence from the
 received text.
6. The system of claim 1, wherein the one or more servers
 are further configured to determine a highest ranking joke
 from among the one or more jokes.

34

7. A method for generating jokes, comprising:
 providing one or more servers communicatively coupled
 with one or more databases;
 in response to receiving text, at the one or more servers
 through a telecommunications network or at a first
 computing device:
 selecting one or more topic keywords from the received
 text using the one or more servers or using the first
 computing device;
 generating one or more punch words with the one or
 more servers using words stored in the one or more
 databases which are related to the one or more topic
 keywords, wherein the one or more punch words are
 based at least in part on one of:
 one or more wordplay scores, and;
 one or more of: data representing common knowl-
 edge, data representing anticipated knowledge of
 an intended audience, data representing beliefs of
 the intended audience, and data representing atti-
 tudes of the intended audience;
 adding one or more bridges to the one or more punch
 words, using the one or more servers, to generate one
 or more jokes, wherein generating the one or more
 jokes comprises generating a punchline; and
 communicating a signal to the first computing device
 through the telecommunications network using the
 one or more servers; and
 in response to receiving the signal at the first computing
 device, displaying or speaking one of the one or more
 jokes using the first computing device.
8. The method of claim 7, wherein the received text is
 received at the one or more servers from the first computing
 device through the telecommunications network.
9. The method of claim 8, wherein the received text is text
 typed or spoken to a virtual assistant using the first com-
 puting device.
10. The method of claim 7, wherein the received text is
 received at the one or more servers from a third party server
 and not from the first computing device.
11. The method of claim 7, further comprising selecting a
 topic sentence from the received text using the one or more
 servers or the first computing device.
12. The method of claim 7 further comprising, using the
 one or more servers, determining a highest ranking joke
 from among the one or more jokes.

* * * * *